



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES
- RABAT

PROJET DE FIN D'ANNÉE

Automatisation de l'analyse de
l'importation de motos d'occasion en
utilisant le web scraping et l'estimation
fiscale

Élèves :
Jad FALAQ

Encadrant :
Mr Mohamed LAZAAR
Mme Oumaima ZAOUI

Jury :
Mr Raddouane CHIHEB
Mme EL AFIA

Année Universitaire 2024-2025

Remerciements

Je tiens à exprimer ma profonde reconnaissance envers toutes les personnes qui ont contribué, de manière directe ou indirecte, à la réalisation de ce projet de fin d'année.

Je remercie tout particulièrement **Monsieur Mohamed LAZAAR** mon encadrant et **Madame Oumaima ZAOUI**, pour leur accompagnement constant, leur disponibilité et leurs conseils avisés tout au long de ce travail. Leur expertise dans le domaine du développement web et du scraping a été un appui essentiel dans la réussite de ce projet, et leur rigueur m'a permis de structurer mes idées avec clarté et efficacité.

Je souhaite également exprimer toute ma gratitude à **Monsieur Abdellatif EL AFIA**, chef de filière, pour sa bienveillance, son écoute et son soutien durant l'ensemble de l'année universitaire. Son engagement auprès des étudiants et sa vision pédagogique ont grandement contribué à l'environnement stimulant et formateur dans lequel ce projet a été mené.

Mes remerciements s'adressent également aux membres du jury, **Pr. Raddouane CHIEB** et **Madame El AFIA**, pour l'intérêt qu'ils ont porté à mon travail, ainsi que pour leurs remarques constructives et enrichissantes. Leurs retours constituent une réelle valeur ajoutée, tant pour l'amélioration de ce projet que pour mon développement personnel et professionnel.

Enfin, je remercie l'ensemble du corps enseignant de l'ENSIAS pour la qualité des enseignements dispensés au cours de ces années, ainsi que mes collègues de promotion pour les échanges, l'entraide et les moments partagés.

Résumé

Ce projet vise à automatiser l'analyse de la rentabilité de l'importation de motos d'occasion depuis l'Europe vers le Maroc, en utilisant des techniques avancées de web scraping et d'estimation des droits d'importation. La première étape consiste à collecter les annonces de motos disponibles sur des plateformes spécialisées, en filtrant les véhicules dont la date de mise en circulation est inférieure à cinq ans, conformément à la réglementation en vigueur.

Chaque annonce est ensuite traitée pour extraire des informations clés telles que la marque, le modèle, le prix, le kilométrage, la date de publication, et le lieu. Ces données sont automatiquement injectées dans le simulateur officiel des droits de douane marocain, permettant d'obtenir une estimation précise des taxes à payer lors de l'importation.

L'outil ainsi développé permet d'identifier rapidement les motos qui présentent une marge bénéficiaire potentielle en tenant compte à la fois du coût d'acquisition et des frais douaniers. Ce système offre une aide à la décision efficace pour les importateurs, en réduisant les risques financiers liés à l'importation et en optimisant le choix des véhicules à importer. Le projet illustre également l'application concrète de méthodes d'intelligence artificielle et d'automatisation dans un contexte économique et réglementaire complexe.

Abstract

This project aims to automate the profitability analysis of importing used motorcycles from Europe to Morocco, using advanced web scraping techniques and customs duty estimation. The first step involves collecting motorcycle listings from specialized platforms, filtering vehicles that have been in circulation for less than five years, in accordance with current regulations.

Each listing is then processed to extract key information such as brand, model, price, mileage, publication date, and location. These data are automatically fed into the official Moroccan customs duty simulator, allowing for an accurate estimation of the taxes payable upon import.

The developed tool quickly identifies motorcycles with potential profit margins by considering both the acquisition cost and customs fees. This system provides effective decision-making support for importers, reducing financial risks and optimizing vehicle selection. The project also demonstrates the practical application of artificial intelligence and automation methods within a complex economic and regulatory framework.

Liste des Abréviations

- API : Application Programming Interface
- CSV : Comma-Separated Values (valeurs séparées par des virgules)
- DOM : Document Object Model
- HTML : HyperText Markup Language
- HTTP : HyperText Transfer Protocol
- HTTPS : HyperText Transfer Protocol Secure
- IP : Internet Protocol
- JSON : JavaScript Object Notation
- MVC : Model-View-Controller
- Selenium : Outil d'automatisation de navigateur web
- URL : Uniform Resource Locator
- VPN : Virtual Private Network
- XPath : XML Path Language (langage de requête pour extraire des données dans XML/HTML)
- UI : User Interface (interface utilisateur)
- UX : User Experience (expérience utilisateur)
- CSV : Comma Separated Values (format de fichier texte tabulaire)
- GUI : Graphical User Interface

Table des matières

Remerciements	1
Résumé	2
Liste des Abréviations	4
1 Introduction	9
1.1 Contexte général	9
1.2 Problématique et motivation	9
1.3 Objectifs	10
1.4 Revue de littérature	10
2 Présentation du projet	11
2.1 Description générale	11
2.2 Enjeux économiques et réglementaires	11
2.3 Synthèse des étapes	12
3 Méthodologie et déroulement technique	13
3.1 Étape 1 : Scraping des motos éligibles en Europe	13
3.2 Étape 2 : Estimation des frais d'importation via le simulateur officiel	13
3.3 Étape 3 : Scraping des prix moyens au Maroc	13
3.4 Étape 4 : Validation et filtrage des annonces	13
3.5 Étape 5 : Compilation des données finales et visualisation	14
4 État de l'art	15
4.1 Web scraping : outils et enjeux	15
4.2 Estimation fiscale des véhicules	16
5 Implémentation technique	17
5.1 Structure générale du script	17
5.1.1 Configuration du navigateur Selenium et chargement des modèles	17
5.1.2 Extraction et Chargement des modèles des marques	19
5.1.3 Normalisation et correspondance approximative	21
5.1.4 Définition de la fonction d'estimation fiscale	21
5.1.5 Boucle principale de scraping des annonces filtrées sur le site leparking-moto.ma	24
5.1.6 Consolidation et export des résultats	25
5.2 Description des données collectées	26
5.3 Analyse de rentabilité	26

5.3.1	Collecte des prix de revente estimés	26
5.3.2	Intégration et enrichissement des données	27
5.3.3	Calcul de la rentabilité	27
5.3.4	Estimation de la valeur locale au Maroc via Avito.ma	27
5.3.5	Exemmmple de Fichier CSV Final	29
5.4	Synthèse	30
5.5	Représentation graphique des marges brutes	30
5.5.1	Structure du graphique	30
5.5.2	Algorithme de génération	31
5.5.3	Interprétation des résultats	31
5.6	Code Python de visualisation	31
6	Description des étapes du scrapping	33
6.1	Saisie des paramètres de recherche	33
6.2	Affichage des résultats de recherche	33
6.3	Navigation vers le calculateur de dédouanement	34
6.4	Sélection du type de véhicule	35
6.5	Sélection de la marque du véhicule	35
6.6	Sélection du modèle du véhicule	36
6.7	Saisie de la date de première mise en circulation	37
6.8	Extraction des frais totaux de dédouanement	38
6.9	Scrapping comparatif des annonces locales (Avito Maroc)	39
6.9.1	Critères de correspondance	39
6.10	Visualisation finale des marges brutes	40
6.10.1	Diagramme de type "bougie" des annonces	40
6.10.2	Lecture du graphique	41
7	Conclusion et perspectives	42
7.1	Perspectives	43
7.1.1	1. Intégration de l'intelligence artificielle pour la prédiction des marges futures	43
7.1.2	Déploiement d'une plateforme web interactive	43
7.1.3	Enrichissement de la base de données avec d'autres marchés	43
7.1.4	Système d'alerte intelligente et scoring	44
7.1.5	Automatisation des démarches administratives	44
7.1.6	Couplage avec une blockchain de traçabilité des véhicules	44
	Conclusion Générale	45

Table des figures

5.1	Exemple d'entrée d'un utilisateur	19
5.2	Exemple d'une partie du fichier tous-les-modeles.csv	21
5.3	Exemple de database pour la recherche "yamaha mt 09"	30
5.4	Visualisation des marges brutes par annonce de moto	31
6.1	Interface de saisie des paramètres de recherche	33
6.2	Page de résultats de recherche sur leparking-moto.ma	34
6.3	Interface du calculateur de dédouanement des douanes marocaines	34
6.4	Sélection du type de véhicule - Motorcycle	35
6.5	Sélection de la marque dans le menu déroulant	36
6.6	Sélection du modèle dans le menu déroulant	36
6.7	Ouverture du calendrier pour la sélection de la date	37
6.8	Sélection de l'année dans le calendrier	37
6.9	Sélection du mois et du jour dans le calendrier	38
6.10	Extraction du montant total des frais de dédouanement	38
6.11	Finalisation du scrapping des annonces en Europe	39
6.12	Scrapping des annonces de la même moto au Maroc	40
6.13	Visualisation des marges brutes des annonces de motos	40

Liste des tableaux

Chapitre 1

Introduction

1.1 Contexte général

L'importation de motos d'occasion représente un secteur économique important au Maroc, répondant à une forte demande locale pour des véhicules accessibles et diversifiés. Cependant, cette activité est soumise à des réglementations strictes, notamment en matière de conformité technique, de contrôle des émissions, et surtout de droits de douane et taxes d'importation. Le calcul précis de ces frais, variable selon la marque, le modèle, l'année de mise en circulation et d'autres caractéristiques du véhicule, est un facteur clé pour évaluer la rentabilité potentielle d'une importation. Par ailleurs, les informations sur les motos proposées sur les marchés européens sont dispersées sur de multiples plateformes web, souvent mises à jour dynamiquement. Dans ce contexte complexe, il devient essentiel de développer des outils automatisés capables de collecter, traiter et analyser efficacement ces données afin d'aider les importateurs à prendre des décisions éclairées. Ce projet s'inscrit dans cette démarche en utilisant des techniques de web scraping avancées couplées à une estimation automatisée des droits douaniers pour faciliter et sécuriser le processus d'importation des motos d'occasion au Maroc.

1.2 Problématique et motivation

Le marché de l'importation des motos d'occasion au Maroc est caractérisé par une forte demande, mais aussi par une complexité réglementaire et fiscale importante. Les importateurs doivent naviguer entre des sources d'informations multiples et souvent disparates, et faire face à des calculs complexes des droits et taxes d'importation, qui dépendent de critères précis tels que la marque, le modèle et la date de première mise en circulation. Cette complexité engendre des risques financiers, notamment lorsque les coûts réels d'importation ne sont pas correctement estimés en amont. Par ailleurs, la collecte manuelle des données depuis les sites européens d'annonces est chronophage et sujette à erreurs. La motivation principale de ce projet est donc de développer un système automatisé qui facilite la collecte de données fiables et la simulation précise des coûts douaniers, afin d'offrir aux importateurs un outil d'aide à la décision efficace, réduisant les risques et optimisant la sélection des motos à importer. Cette approche allie intelligence artificielle et automatisation pour répondre à un besoin concret et stratégique du marché marocain.

1.3 Objectifs

- Mettre en place un système de web scraping capable de collecter efficacement les annonces de motos disponibles sur des plateformes en ligne, en ciblant précisément la marque, le modèle et le pays européen.
- Valider et filtrer les données recueillies afin de ne conserver que les annonces comportant des informations complètes et fiables, notamment une date de mise en circulation récente conforme aux exigences réglementaires
- Extraire automatiquement les modèles autorisés par la Douane marocaine et construire une base de données référentielle .
- Automatiser l'interaction avec le simulateur officiel des droits de douane afin d'estimer les frais totaux liés à l'importation pour chaque moto candidate.
- Consolider toutes ces données dans un fichier exploitable pour permettre une évaluation rapide et précise de la rentabilité potentielle des importations, offrant ainsi une aide à la décision aux importateurs.

1.4 Revue de littérature

Le web scraping est une technique largement utilisée pour extraire automatiquement des données de sites web afin d'alimenter des analyses ou des applications. Parmi les outils les plus populaires, Selenium se distingue par sa capacité à interagir avec des pages dynamiques générées par JavaScript, ce qui est essentiel pour scraper des sites modernes. BeautifulSoup, quant à elle, est une bibliothèque efficace pour parser des contenus HTML statiques mais limitée face aux contenus dynamiques. Plusieurs études ont démontré l'efficacité de Selenium dans des projets complexes nécessitant l'automatisation de navigation web, le remplissage de formulaires, et la gestion d'interactions utilisateur simulées.

Dans le domaine fiscal, l'automatisation des calculs de droits et taxes grâce à des simulateurs en ligne devient une pratique de plus en plus courante, notamment pour faciliter la conformité réglementaire et optimiser les coûts. Cependant, l'intégration de données collectées via scraping avec ces simulateurs nécessite des méthodes robustes pour gérer les correspondances approximatives et les erreurs potentielles. Plusieurs travaux ont également mis en avant l'importance de la qualité des données extraites, car des informations incomplètes ou erronées peuvent biaiser les calculs et décisions.

Ce projet s'appuie donc sur ces avancées pour combiner collecte automatisée de données en ligne et estimation fiscale, offrant ainsi une solution innovante adaptée au contexte spécifique de l'importation de motos d'occasion au Maroc.

Chapitre 2

Présentation du projet

2.1 Description générale

Ce projet a pour but de développer une chaîne automatisée permettant de collecter, traiter et analyser des données relatives à l'importation de motos d'occasion depuis l'Europe vers le Maroc. Il s'appuie sur l'utilisation de techniques de web scraping pour extraire des annonces détaillées disponibles sur des plateformes spécialisées, ainsi que sur l'automatisation de l'interaction avec le simulateur officiel des droits de douane marocain. Cette approche permet de calculer précisément les coûts d'importation pour chaque moto sélectionnée, en tenant compte des caractéristiques spécifiques telles que la marque, le modèle et la date de première mise en circulation.

Pour affiner l'analyse de rentabilité, chaque annonce traitée est également comparée aux annonces locales disponibles sur le marché marocain via la plateforme Avito.ma. Le système scrappe automatiquement les annonces similaires correspondant au même modèle et à la même année de mise en circulation, permettant ainsi d'estimer un prix moyen de revente au Maroc. En croisant ce prix estimé avec le coût total d'acquisition.

Le système ainsi conçu offre aux importateurs un outil performant pour évaluer rapidement la rentabilité potentielle de chaque véhicule, en intégrant à la fois les données du marché européen et les contraintes fiscales et commerciales locales.

2.2 Enjeux économiques et réglementaires

L'importation de motos d'occasion au Maroc s'inscrit dans un contexte économique dynamique, où la demande pour des véhicules abordables et diversifiés est croissante. Toutefois, cette activité est encadrée par un ensemble de réglementations strictes, notamment en ce qui concerne les normes techniques, les critères d'émissions polluantes, et les formalités douanières. Les droits de douane, la TVA, ainsi que d'autres taxes spécifiques, peuvent fortement influencer le coût final d'acquisition des motos importées, impactant directement leur compétitivité sur le marché local. Par ailleurs, le respect des normes et des procédures administratives est essentiel pour éviter des sanctions et garantir la conformité légale. Ces enjeux imposent aux importateurs une gestion rigoureuse et une évaluation précise des coûts associés, afin d'assurer la rentabilité des opérations. Le projet vise donc à apporter une solution automatisée facilitant ces évaluations, tout en tenant compte des contraintes économiques et réglementaires spécifiques au contexte marocain.

2.3 Synthèse des étapes

Le projet s'articule autour de plusieurs étapes clés permettant d'automatiser l'analyse de rentabilité des motos d'occasion importables :

1. **Scraping des annonces** : collecte automatisée des annonces de motos sur des plateformes européennes, selon la marque et le modèle spécifiés par l'utilisateur.
2. **Validation des données** : filtrage des annonces pour ne conserver que celles présentant des informations complètes et une date de première mise en circulation récente (moins de 5 ans).
3. **Collecte des modèles officiels** : extraction des modèles autorisés à l'importation depuis le site officiel de la Douane marocaine, constituant une base référentielle.
4. **Correspondance des modèles** : application d'une méthode de correspondance approximative pour aligner les modèles utilisateurs avec ceux reconnus par la Douane.
5. **Estimation des droits et taxes** : automatisation de la saisie des caractéristiques dans le simulateur douanier et extraction des coûts totaux d'importation.
6. **Scraping des prix de revente au Maroc** : pour chaque moto validée, le système interroge automatiquement la plateforme Avito.ma pour collecter les annonces similaires correspondant à la même année et au même modèle, et calcule un prix moyen de revente local.
7. **Calcul de la rentabilité** : à partir du prix d'achat en Europe et des frais d'importation, le système calcule la rentabilité brute de chaque moto, exprimée en valeur absolue et en pourcentage.
8. **Compilation finale** : consolidation de toutes les données dans un fichier CSV synthétique, facilitant l'analyse, le tri et la prise de décision.

Chapitre 3

Méthodologie et déroulement technique

Ce projet s'articule autour de cinq étapes majeures, organisées de manière séquentielle afin d'automatiser l'analyse de rentabilité des motos d'occasion importées depuis l'Europe vers le Maroc.

3.1 Étape 1 : Scraping des motos éligibles en Europe

Dans un premier temps, le système effectue une collecte automatisée des annonces de motos d'occasion disponibles sur des plateformes européennes. Il ne conserve que les annonces correspondant à des modèles éligibles à l'importation selon la base officielle de la Douane marocaine. Cette étape garantit que seules les motos conformes à la réglementation sont prises en compte dès le début du processus.

3.2 Étape 2 : Estimation des frais d'importation via le simulateur officiel

Pour chaque moto éligible, le système interagit automatiquement avec le simulateur en ligne de la Douane marocaine. Il y insère les informations pertinentes (marque, modèle, date de première mise en circulation) et en extrait le montant total des frais d'importation (droits, TVA, autres taxes). Cette estimation est cruciale pour établir le coût réel d'acquisition d'une moto importée.

3.3 Étape 3 : Scraping des prix moyens au Maroc

Ensuite, le système effectue une recherche ciblée sur Avito.ma afin de collecter les annonces locales correspondant au même modèle et à la même année de mise en circulation. Il en déduit un prix moyen de revente estimé au Maroc. Cette étape permet de croiser les données d'importation avec le contexte réel du marché marocain.

3.4 Étape 4 : Validation et filtrage des annonces

Une phase de validation est ensuite effectuée pour éliminer les annonces incomplètes ou incohérentes. Les critères retenus incluent notamment la date de première mise en circulation (inférieure à cinq ans), le kilométrage, le prix, et le lieu d'origine. Cette étape garantit la qualité et la pertinence du jeu de données utilisé dans les calculs.

3.5 Étape 5 : Compilation des données finales et visualisation

Enfin, toutes les données validées sont compilées dans un fichier CSV synthétique. Celui-ci contient pour chaque moto : le prix d'achat, les frais d'importation, le prix moyen de revente estimé, et les marges calculées. Une visualisation graphique permet de distinguer rapidement les motos rentables (marge positive) de celles à éviter (marge négative), offrant ainsi un outil d'aide à la décision simple et efficace.

Chapitre 4

État de l'art

4.1 Web scraping : outils et enjeux

Le web scraping est une technique d'extraction automatique de données à partir de sites web, utilisée pour collecter des informations structurées disponibles publiquement. Parmi les outils les plus courants figurent Selenium et BeautifulSoup. Selenium permet d'automatiser la navigation sur des sites web dynamiques, notamment ceux utilisant intensivement JavaScript, en contrôlant un navigateur réel ou simulé. BeautifulSoup est une bibliothèque Python facilitant l'analyse et le parsing de documents HTML statiques. Malgré leur efficacité, le web scraping fait face à plusieurs limitations légales et techniques. Les fichiers robots.txt définissent les règles d'accès aux contenus par les robots, et certains sites mettent en place des systèmes de protection tels que les CAPTCHA pour prévenir l'automatisation abusive. De plus, un scraping intensif peut engendrer une surcharge des serveurs et violer les conditions d'utilisation, exposant les utilisateurs à des sanctions.

Dans le cadre de ce projet, Selenium a été privilégié pour sa capacité à gérer les pages dynamiques et les interactions complexes nécessaires à l'extraction fiable des données. Un défi particulier rencontré a été l'insertion de la date de première mise en circulation dans le simulateur en ligne de la Douane marocaine. Ce champ utilise un calendrier interactif, rendant impossible une saisie textuelle directe. Le script automatise alors la navigation dans ce calendrier, sélectionnant année, mois et jour afin d'assurer la bonne saisie de la date. En revanche, la présence de CAPTCHA sur certains sites a représenté une limite importante. Ne disposant pas des moyens payants nécessaires à leur résolution automatisée, une solution alternative a été adoptée : le choix d'un site sans CAPTCHA, permettant ainsi un scraping fluide et sans interruption.

4.2 Estimation fiscale des véhicules

L'estimation des droits et taxes applicables à l'importation de motos d'occasion au Maroc constitue une étape cruciale pour évaluer la rentabilité potentielle. Cette estimation repose sur un simulateur officiel accessible en ligne, développé par l'administration des Douanes marocaines. Le simulateur permet de calculer les montants des droits de douane, de la TVA, ainsi que d'autres frais réglementaires, en fonction de paramètres précis tels que la marque, le modèle et la date de première mise en circulation du véhicule. L'utilisation de cet outil en mode automatisé dans notre projet nécessite une interaction programmée avec son interface, notamment la sélection dynamique des valeurs dans des listes déroulantes et la saisie d'une date via un calendrier interactif. Cette automatisation garantit une extraction fiable et rapide des coûts fiscaux, facilitant ainsi l'analyse globale de la rentabilité pour chaque moto importée. Ce processus contribue à réduire les erreurs humaines et accélère considérablement la prise de décision pour les importateurs.

Chapitre 5

Implémentation technique

5.1 Structure générale du script

Le script Python développé pour ce projet est structuré en plusieurs modules fonctionnels distincts afin d'assurer une architecture claire et modulaire. Il comprend notamment les étapes suivantes :

1. **Configuration du navigateur Selenium** : mise en place des options nécessaires pour une navigation optimisée et sécurisée.
2. **Extraction et Chargement des modèles des marques** : récupération automatique des modèles disponibles pour chaque marque depuis le site de la Douane marocaine, et l'importation des données extraites dans le script sous forme de fichier CSV, servant de référence pour la validation des annonces.
3. **Normalisation et correspondance approximative** : gestion des écarts de nommage entre les modèles saisis par l'utilisateur et ceux présents dans la base douanière.
4. **Définition de la fonction d'estimation fiscale** : automatisation de l'interaction avec le simulateur douanier pour extraire la valeur totale des droits et taxes d'importation.
5. **Boucle principale de scraping** : collecte des annonces de motos selon les critères définis, avec application de filtres rigoureux sur les données extraites et utilisation de la fonction d'estimation fiscale.
6. **Consolidation et export des résultats** : sauvegarde des données finales dans un fichier CSV pour exploitation ultérieure.

5.1.1 Configuration du navigateur Selenium et chargement des modèles

Le script commence par importer les bibliothèques nécessaires, notamment Selenium pour l'automatisation du navigateur, Pandas pour la gestion des données, ainsi que d'autres utilitaires pour le traitement et la gestion des erreurs.

La configuration du navigateur Chrome via Selenium comprend plusieurs options destinées à simuler un comportement humain et à contourner certaines protections web :

- Désactivation des erreurs SSL et des avertissements de certificat.
- Autorisation de contenu local non sécurisé.
- Désactivation des protections web pour éviter les blocages.
- Configuration d'un user-agent personnalisé pour simuler un navigateur réel.
- Blocage des demandes de géolocalisation.

Le fichier CSV `tous_les_modeles.csv` est chargé avec Pandas pour fournir la liste des modèles de motos autorisés.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException, NoSuchElementException
.
.
.

import traceback
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from datetime import date, timedelta

# ==== CONFIGURATION SELENIUM ====
options = Options()
options.add_argument("--window-size=1280x720")
options.add_argument("--ignore-certificate-errors")
.
.
.

options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)
↳ AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36")
options.add_experimental_option("prefs", {
    "profile.default_content_setting_values.geolocation": 2
})

df_modeles = pd.read_csv("tous_les_modeles.csv")

def normalize(text):
    return re.sub(r"[^\w]", "", text.lower())
    # ==== CHROME DRIVER ====
service = Service(r"C:\chromedriver-win64\chromedriver.exe")
driver = webdriver.Chrome(service=service, options=options)

# ==== DEMANDER LA MOTO ET PAGES ====
marque = input("Entrez la marque de la moto (ex: yamaha) : ").strip().lower()
modele = input("Entrez le modèle de la moto (ex: mt 07) : ").strip().lower()
moto = marque + " " + modele
moto_url = moto.replace(" ", "-")
nb_pages = int(input("Combien de pages veux-tu scraper ? : "))

pays=input("Entrer le pays voulue : ").strip().lower()

url = f"https://www.leparking-moto.ma/moto-occasion/{pays}/{moto_url}.html"
print(f"Navigation vers : {url}")

driver.get(url)
time.sleep(3)

data = []
page_count = 0
```

```
# ==== DATE LIMITE (aujourd'hui - 5 ans) ====
date_limite = datetime.date.today() - datetime.timedelta(days=5*365)
```

```
DevTools listening on ws://127.0.0.1:57351/devtools/browser/a91323c2-4905-4671-b5e3-1dcad201d37c
Entrez la marque de la moto (ex: yamaha) : yamaha
Entrez le modèle de la moto (ex: mt 07) : mt 09
Combien de pages veux-tu scraper ? : 1
Entrer le pays voulue : Espagne
Navigation vers : https://www.leparking-moto.ma/moto-occasion/espagne/yamaha-mt-09.html
```

FIGURE 5.1 – Exemple d’entrée d’un utilisateur

5.1.2 Extraction et Chargement des modèles des marques

Pour valider les annonces scrappées, il est nécessaire de disposer d’une liste fiable des modèles de motos autorisés à l’importation au Maroc. Cette liste est extraite automatiquement depuis le site officiel du simulateur de la Douane marocaine via un script Selenium.

Le script suit les étapes suivantes :

- Accès à la page du simulateur douanier.
- Ouverture du menu déroulant des marques et récupération de toutes les marques disponibles.
- Pour chaque marque, sélection dans le menu et ouverture du sous-menu des modèles.
- Extraction des modèles disponibles pour la marque sélectionnée.
- Écriture des paires *marque-modèle* dans un fichier CSV nommé `tous_les_modeles.csv`.
- Gestion des cas où certaines marques n’ont pas de modèles listés.
- Gestion des fenêtres modales pouvant bloquer l’accès aux menus.

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
import csv
import time

chemin_chromedriver = r"C:\chromedriver-win64\chromedriver.exe"
url = "https://www2.douane.gov.ma/mcv/#/view?pageId=2"

options = Options()
options.add_argument("--start-maximized")

driver = webdriver.Chrome(service=Service(chemin_chromedriver), options=options)
wait = WebDriverWait(driver, 15)

driver.get(url)

# Ouvrir le menu des marques
wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
    ↪ "mat-select[formcontrolname='marque']"))).click()
time.sleep(1)
```

```
# Récupérer toutes les marques disponibles
wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, "mat-option span")))
marque_elements = driver.find_elements(By.CSS_SELECTOR, "mat-option span")
marques = [m.text.strip() for m in marque_elements if m.text.strip()]

print(f" {len(marques)} marques détectées.")

with open("tous_les_modeles.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Marque", "Modèle"])

    for marque in marques:
        try:
            # Fermer backdrop s'il est présent
            try:
                backdrop = driver.find_element(By.CSS_SELECTOR,
                    ↪ ".cdk-overlay-backdrop")
                driver.execute_script("arguments[0].click();", backdrop)
                time.sleep(0.5)
            except:
                pass

            # Réouvrir le menu des marques
            wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
                ↪ "mat-select[formcontrolname='marque']"))).click()
            time.sleep(1)

            .
            .
            .

            for modele in modeles:
                writer.writerow([marque, modele])

        except Exception as e:
            print(f" Problème avec {marque} : {e}")
            continue

print(" Export terminé → fichier : tous_les_modeles.csv")
driver.quit()
```

Après avoir extrait et exporté les modèles dans un fichier CSV, le script principal commence par charger ces données dans un DataFrame Pandas. Ce chargement permet de disposer en mémoire d'une liste complète et à jour des modèles de motos autorisés à l'importation, qui servira ensuite de référence pour valider et corriger les annonces récupérées lors du scraping.

La commande principale utilisée est la suivante :

```
import pandas as pd

df_modeles = pd.read_csv("tous_les_modeles.csv")
```

Cette étape est essentielle pour que la fonction de correspondance approximative

```

tutp > tous_les_modeles.csv > data
937
938 YAMAHA,MTX 20 / NAKED (<= 20 CC)
939 YAMAHA,BWS 50 / 12P / NAKED / NG / SA231 (<= 50 CC)
940 YAMAHA,DT 50 / supermotard (<= 50 CC)
941 YAMAHA,Giggle (<= 50 CC)
942 YAMAHA,Jogg R (<= 50 CC)
943 YAMAHA,MATE / NEW MATE / V50 (<= 50 CC)
944 YAMAHA,NEOS 50 / NEOS 50 4T / 4T UBS/ VINO 50/ YMS0F (<= 50 CC)
945 YAMAHA,Slider 80 Naked (<= 50 CC)
946 YAMAHA,TZR 50 (<= 50 CC)
947 YAMAHA,1000 MT-10 / SP (> 80 CC)
948 YAMAHA,1000 YZF R1 (> 80 CC)
949 YAMAHA,1000 YZF R1 M (> 80 CC)
950 YAMAHA,110 TT-R (> 80 CC)
951 YAMAHA,1100 XVS DRAGSTAR (> 80 CC)
952 YAMAHA,1200 XTZ Super Tenere (> 80 CC)
953 YAMAHA,125 D'Elight (> 80 CC)
954 YAMAHA,125 MT (> 80 CC)
955 YAMAHA,125 NMAX/ GPD125-A (> 80 CC)
956 YAMAHA,125 TRICITY / MWS (> 80 CC)
957 YAMAHA,125 XMAX TECH MAX/ YP 125 R XMAX/ YP 125 RA SP (> 80 CC)
958 YAMAHA,125 X-MAX/ XMAX IRON MAX (> 80 CC)
959 YAMAHA,125 YBR (> 80 CC)
960 YAMAHA,125 YS/ XENTER (> 80 CC)
961 YAMAHA,125 YZ (> 80 CC)
962 YAMAHA,125 YZF R (10) (> 80 CC)
963 YAMAHA,1300 FJR / A /AE/AS/GT (> 80 CC)
964 YAMAHA,1900 XV19 STAR VENTURE (> 80 CC)
965 YAMAHA,250 XT (> 80 CC)
966 YAMAHA,250 YZ/ YZ 250 F/EX/X/WR 250 F (> 80 CC)
967 YAMAHA,300 MT-03 (> 80 CC)
968 YAMAHA,300 TRICITY/ MWD300 (> 80 CC)
969 YAMAHA,300 X-MAX TECH MAX/ IRON MAX/ CZD300-A SP (> 80 CC)
970 YAMAHA,300 X-MAX/250/ CZD300-A (> 80 CC)
971 YAMAHA,300 YZF R3 (> 80 CC)
972 YAMAHA,400 X-Max (> 80 CC)

```

FIGURE 5.2 – Exemple d’une partie du fichier tous-les-modeles.csv

5.1.3 Normalisation et correspondance approximative

Afin de gérer les écarts de nommage entre les modèles saisis par l'utilisateur et ceux présents dans la base douanière, une fonction de correspondance approximative a été développée. Cette fonction normalise les chaînes de caractères en supprimant les caractères non alphanumériques et en convertissant le texte en minuscules pour faciliter la comparaison.

La fonction `trouver_modele_proche` effectue une recherche dans la liste des modèles disponibles pour une marque donnée, en tentant d'identifier un modèle dont la chaîne normalisée est partiellement contenue dans celle fournie par l'utilisateur, ou inversement. Cette méthode permet de pallier les variations d'appellations et les différences mineures d'orthographe.

```

def trouver_modele_proche(modele_scrape, marque_scrape, df_modeles):
    modele_norm = normalize(modele_scrape)
    subset = df_modeles[df_modeles["Marque"].str.lower() == marque_scrape.lower()]
    modeles_dispo = subset["Modèle"].dropna().unique()

    # Correspondance manuelle partielle
    for modele in modeles_dispo:
        if normalize(modele_norm) in normalize(modele):
            return modele
        if normalize(modele) in normalize(modele_norm):
            return modele

    return None

```

5.1.4 Définition de la fonction d'estimation fiscale

Cette fonction automatise l'interaction avec le simulateur officiel des droits de douane marocain. Elle prend en entrée la marque, le modèle et la date de première mise en circulation de la moto, puis navigue sur le site pour sélectionner les options correspondantes.

Les étapes principales sont les suivantes :

- Accès à la page du simulateur et navigation vers la section « Calcul des droits ».
- Sélection du type de véhicule (« Motorcycle »).
- Sélection de la marque et du modèle parmi les options proposées.
- Ouverture du calendrier pour saisir la date de première mise en circulation, avec gestion dynamique de la navigation pour sélectionner l'année, le mois et le jour.
- Validation des informations et extraction automatique de la valeur totale des droits et taxes d'importation.
- Gestion des exceptions et fermeture propre du navigateur.

Cette fonction permet d'obtenir une estimation précise et automatisée des coûts fiscaux liés à l'importation de chaque moto, essentielle pour le calcul de la rentabilité.

```
def get_valeur_taxable_total(marque, modele, date_str):

    service = Service(r"C:\chromedriver-win64\chromedriver.exe")
    options = Options()
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(service=service, options=options)
    wait = WebDriverWait(driver, 15)

    try:
        jour, mois, annee = date_str.split("/")
        mois_map = {
            "01": "JANV.", "02": "FÉVR.", "03": "MARS", "04": "AVR.", "05": "MAI",
            "06": "JUIN", "07": "JUIL.", "08": "AOÛT", "09": "SEPT.",
            "10": "OCT.", "11": "NOV.", "12": "DÉC."
        }
        mois_nom = mois_map[mois]

        driver.get("https://www2.douane.gov.ma/mcv/#/view?pageId=2")
        time.sleep(2)

        # Clic sur "Calcul des droits"
        onglets = wait.until(EC.presence_of_all_elements_located(
            (By.CSS_SELECTOR,
             ↪ "div.b-detail__main-info-characteristics-one.calculator")))
        for onglet in onglets:
            if "CALCUL DES DROITS" in onglet.text.upper():
                driver.execute_script("arguments[0].click();", onglet)
                break
        time.sleep(1.5)

        # Clic sur "Motorcycle"
        moto_svg = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR,
            ↪ "label[for='type1']")))
        driver.execute_script("arguments[0].click();", moto_svg)
        time.sleep(1.5)

        # Sélection de la marque
        wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
            ↪ "mat-select[formcontrolname='marque']"))).click()
        time.sleep(1)
        marques = wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR,
            ↪ "mat-option span")))
        for m in marques:
```

```

        if m.text.strip().upper() == marque.upper():
            m.click()
            break
    time.sleep(1)

    # Sélection du modèle
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
        ↪ "mat-select[formcontrolname='modele']"))).click()
    time.sleep(1)
    modeles = wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR,
        ↪ "mat-option span"))))
    for mod in modeles:
        if modele.lower() in mod.text.strip().lower():
            mod.click()
            break
    time.sleep(1)

    # Calendrier : ouverture et sélection de la date
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
        ↪ "button[aria-label='Open calendar']"))).click()
    time.sleep(1)

    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
        ↪ "button.mat-calendar-period-button"))).click()
    time.sleep(1)

    # Sélection de l'année avec navigation si nécessaire
    while True:
        .
        .
        .
        .

    # Extraction de la valeur taxable
    elements = driver.find_elements(By.CSS_SELECTOR,
        ↪ "div.b-compare__block-inside-value")
    montant = None
    for el in reversed(elements):
        texte = el.text.strip()
        if "DH" in texte:
            montant_str = re.sub(r"[^\d]", "", texte)
            if montant_str:
                montant = int(montant_str)
                break

    return montant

except Exception as e:
    print(" Erreur :", repr(e))
    traceback.print_exc()
    return None

finally:
    time.sleep(3)
    driver.quit()

```


5.1.5 Boucle principale de scraping des annonces filtrées sur le site leparking-moto.ma

Cette étape consiste à collecter les annonces de motos disponibles sur le site ciblé selon les critères définis par l'utilisateur (marque, modèle, pays, nombre de pages). Le script applique des filtres rigoureux afin de ne conserver que les annonces pertinentes et récentes, et utilise la fonction d'estimation fiscale pour chaque moto validée.

Les principales étapes de cette boucle sont :

- Définition d'une date limite correspondant à aujourd'hui moins cinq ans, pour ne garder que les motos récentes.
- Pour chaque page, extraction de la liste des annonces.
- Pour chaque annonce, extraction des informations suivantes :
 - Prix, nettoyé des caractères non numériques.
 - Modèle, avec correction par correspondance approximative.
 - Lien vers l'annonce.
 - Kilométrage et année de mise en circulation.
 - Lieu, filtré pour correspondre au pays cible.
 - Date de publication complète, construite à partir de l'année, du mois et du jour.
- Vérification que la date complète est supérieure à la date limite.
- Appel à la fonction d'estimation fiscale pour obtenir la valeur totale des droits et taxes.
- Ajout des données valides dans une liste pour exploitation ultérieure.
- Navigation vers la page suivante tant que le nombre de pages fixé n'est pas atteint.

```
data = []
page_count = 0

# Date limite (aujourd'hui - 5 ans)
date_limite = datetime.date.today() - datetime.timedelta(days=5*365)

while page_count < nb_pages:
    try:
        annonces = WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CSS_SELECTOR, "li.li-result"))
        )

        for annonce in tqdm(annonces, desc=f"Scraping page {page_count + 1}/{nb_pages}"):
            try:
                # Extraction du prix
                prix_elements = annonce.find_elements(By.CSS_SELECTOR,
                    "div.price-block p.prix")
                prix_brut = None
                for prix_element in prix_elements:
                    texte = prix_element.text.strip()
                    if texte:
                        prix_brut = texte
                        break

                if not prix_brut:
                    continue

                prix_nettoye = re.sub(r"[^\d]", "", prix_brut)
```

```
if prix_nettoye:
    prix_num = int(prix_nettoye)
else:
    continue

# Extraction du modèle et titre
try:
    modele_ = annonce.find_element(By.CSS_SELECTOR,
    ↪ "span.sub-title.title-block
    ↪ span.nowrap.title-block").text.strip()
    modele__ = trouver_modele_proche(modele_, marque.upper(),
    ↪ df_modeles)
    titre = f"{marque} {modele_}"
except:
    titre = "Non précisé"

# Extraction du lien
.
.
.
.

# Ajout des données extraites
data.append({
    "Titre": titre,
    "Prix (DH)": prix_num,
    "Kilométrage (km)": kilometre,
    "Lieu": lieu,
    "Lien": lien,
    "Date complète (jour/mois/année)": date_circulation,
    "Valeur taxable": get_valeur_taxable_total(marque, modele__,
    ↪ date_circulation)
})

except Exception as e:
    print(f"Erreur sur une annonce : {e}")

# Passage à la page suivante
try:
    next_button = driver.find_element(By.CSS_SELECTOR, "li.btn-next a")
    driver.execute_script("arguments[0].click();", next_button)
    time.sleep(3)
    page_count += 1
except Exception as e:
    print("Pas de bouton suivant ou clic impossible :", e)
    break

except Exception as e:
    print(f"Erreur pendant le scraping : {e}")
    break
```

5.1.6 Consolidation et export des résultats

Après la collecte et le filtrage des annonces, les données extraites sont consolidées et préparées pour une utilisation ultérieure. Cette étape inclut :

- La normalisation des liens des annonces (suppression des espaces superflus et conversion en minuscules) afin d'assurer une comparaison cohérente.
- La suppression des doublons basés sur l'URL de l'annonce pour éviter la redondance dans le fichier final.
- La création d'un DataFrame Pandas regroupant toutes les annonces validées.
- L'export des données nettoyées dans un fichier CSV, facilitant l'analyse ou l'intégration dans d'autres outils.

Le code suivant illustre ces opérations :

```
for d in data:
    d["Lien"] = d["Lien"].strip().lower() # Normalisation du lien

df = pd.DataFrame(data)
df.drop_duplicates(subset="Lien", inplace=True)

df.to_csv(f"resultats_motos_{moto_url}.csv", index=False)
print(f" Export terminé : resultats_motos_{moto_url}.csv")
```

5.2 Description des données collectées

Les données collectées proviennent des annonces de motos d'occasion disponibles sur des plateformes européennes, ainsi que des estimations fiscales fournies par le simulateur de la Douane marocaine. Chaque enregistrement contient les informations suivantes :

- **Titre** : combinaison de la marque et du modèle de la moto.
- **Kilométrage (km)** : distance parcourue indiquée pour la moto.
- **Lieu** : pays de localisation de l'annonce.
- **Lien** : URL vers l'annonce originale.
- **Date complète (jour/mois/année)** : date de première mise en circulation, reconstruite à partir des informations fournies.
- **Prix (DH)** : prix de vente annoncé en dirhams marocains.
- **Valeur taxable** : estimation automatique des droits et taxes d'importation en dirhams, extraite du simulateur officiel.

Ces données structurées permettent une analyse approfondie de la rentabilité potentielle des motos importables, en tenant compte à la fois du prix d'achat, des frais d'importation, et des caractéristiques techniques et temporelles des véhicules.

5.3 Analyse de rentabilité

L'objectif de cette phase est d'évaluer la rentabilité potentielle de chaque moto importable en comparant le coût total d'acquisition, incluant le prix d'achat en Europe et les frais douaniers, au prix de revente estimé sur le marché marocain. Cette analyse aide à identifier les motos susceptibles de générer une marge bénéficiaire intéressante.

5.3.1 Collecte des prix de revente estimés

Pour estimer le prix de revente potentiel au Maroc, un script automatisé utilise Selenium afin de scraper les annonces similaires sur le site *Avito Maroc*. Ce site de petites

annonces est une source fiable pour obtenir les prix courants des motos d'occasion sur le marché local.

Le script suit la démarche suivante :

- Construction dynamique de l'URL de recherche en fonction du modèle et de l'année de la moto.
- Navigation automatique sur la page de résultats avec Selenium.
- Gestion des éventuelles fenêtres contextuelles telles que les popups de cookies.
- Extraction des prix affichés dans les annonces, en filtrant les valeurs aberrantes (prix inférieurs à 10 000 DH ou supérieurs à 200 000 DH).
- Calcul de la moyenne des prix extraits pour obtenir un prix moyen représentatif.
- Limitation du nombre d'annonces analysées à 10 pour optimiser le temps d'exécution.

5.3.2 Intégration et enrichissement des données

Le script charge ensuite le fichier CSV contenant les annonces européennes collectées lors des phases précédentes. Pour chaque moto :

- L'année de mise en circulation est extraite et vérifiée afin d'exclure les données futuristes ou erronées.
- Le modèle est isolé du titre en retirant la marque pour permettre une recherche plus précise sur Avito.
- Le script vérifie si un prix moyen pour ce modèle et cette année a déjà été calculé afin d'éviter des requêtes redondantes et de gagner en efficacité.
- Si aucun prix moyen n'est disponible, une nouvelle recherche est effectuée sur Avito.
- Les résultats (prix moyen et nombre d'annonces) sont ajoutés au dataframe dans des colonnes dédiées.

5.3.3 Calcul de la rentabilité

Une fois les prix moyens marocains intégrés, le script procède aux calculs suivants :

- Création d'une colonne **Prix d'achat avec frais** qui correspond à la somme du prix d'achat initial et des droits de douane estimés.
- Calcul de la marge brute potentielle en soustrayant ce prix d'achat total du prix moyen de revente estimé.
- Calcul du pourcentage de rentabilité, qui exprime la marge en pourcentage du coût total d'acquisition.

Les résultats sont enfin sauvegardés dans un nouveau fichier CSV intitulé `resultats_avec_prix_maroc.csv` permettant une exploitation ultérieure.

5.3.4 Estimation de la valeur locale au Maroc via Avito.ma

Afin d'évaluer la rentabilité potentielle de chaque moto importée, il est essentiel de comparer son coût total d'importation avec sa valeur de revente sur le marché marocain. Pour ce faire, une fonction spécifique a été développée :

```
driver = webdriver.Chrome(service=service, options=options)

def scraper_prix_avito(modele, annee):
```

```

url = f"https://www.avito.ma/fr/maroc/motos/{modele.lower().replace('
↳ ', '_')}}--%C3%A0_vendre?regdate={annee}-{annee}"
print(f"URL: {url}")
driver.get(url)

prix_list = []

try:
    # Accepter les cookies si popup
    try:
        cookie_btn = WebDriverWait(driver, 5).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(.,
↳ 'Accepter') or contains(., 'J\'accepte')]"))
        )
        cookie_btn.click()
        print("Cookies acceptés")
        time.sleep(1)
    except:
        print("Pas de popup cookies")
        pass

        .
        .
        .
        .

        prix_list.append(prix)
    except NoSuchElementException:
        print("Élément prix non trouvé dans une annonce")
        continue
    except Exception as e:
        print(f"Erreur extraction prix : {e}")
        continue

except Exception as e:
    print(f"Erreur lors du scraping : {e}")
    return None, 0

if prix_list:
    avg_price = sum(prix_list) / len(prix_list)
    return avg_price , len(prix_list)
else:
    return None, 0

# Chargement du CSV des motos extraites précédemment
df = pd.read_csv("resultats_motos_{marque}-{modele}.csv")

# Préparation des colonnes pour le prix moyen et nombre d'annonces
df['Prix Moyen Maroc'] = None
df['Nombre Annonces'] = 0
annee_courante = dt.now().year

for index, row in df.iterrows():
    try:
        date_str = row['Date complète (jour/mois/année)']
        annee = dt.strptime(date_str, '%d/%m/%Y').year

```

```
if annee > annee_courante:
    print(f" Année {annee} dans le futur - passage à la suivante")
    continue

titre = row['Titre']

.
.
.
.

# Sauvegarde finale
df.to_csv("resultats_avec_prix_maroc.csv", index=False)

# Résumé statistique
if 'Prix Moyen Maroc' in df.columns and df['Prix Moyen Maroc'].notna().any():
    print("\nRésumé statistique :")
    print(f"- Prix moyen au Maroc : {df['Prix Moyen Maroc'].mean():.2f} DH")
    print(f"- Marge moyenne : {df['Marge'].mean():.2f} DH")
    print(f"- Rentabilité moyenne : {df['Rentabilité %'].mean():.2f} %")
else:
    print("\n Aucune donnée de prix disponible pour calculer les statistiques")

print("\n Données sauvegardées dans resultats_avec_prix_maroc.csv")

df = pd.read_csv("resultats_avec_prix_maroc.csv")

# Calculer la marge brute si pas déjà présente
if 'Marge' not in df.columns or df['Marge'].isnull().all():
    df['Prix d\'achat avec frais'] = df['Prix (DH)'] + df['Valeur taxable']
    df['Marge'] = df['Prix Moyen Maroc'] - df['Prix d\'achat avec frais']

# Filtrer les annonces valides avec marge calculée
df_valid = df.dropna(subset=['Marge', 'Prix Moyen Maroc', 'Prix d\'achat avec
↪ frais']).reset_index(drop=True)
```

5.3.5 Exemmple de Fichier CSV Final

5.4 Synthèse

Ce script forme une chaîne automatisée complète, couvrant l'ensemble du processus depuis la collecte des données brutes sur un site d'annonces jusqu'à l'évaluation financière précise de l'importation. Il intègre des mécanismes de validation technique, notamment la vérification des modèles admissibles selon la base douanière, ainsi que des contraintes temporelles en filtrant les motos selon leur ancienneté.

Grâce à cette automatisation, les utilisateurs disposent d'un outil efficace pour identifier rapidement les motos d'occasion présentant un potentiel de rentabilité à l'importation vers le Maroc, réduisant ainsi les risques financiers et optimisant la prise de décision commerciale.

```

tutp > resultats_motos_yamaha-mt-09.csv > L data
1 Titre,Kilométrage (km),Lieu,Lien,Date complète (jour/mois/année),Prix (DH),Valeur taxable
2 yamaha MT 09,15550.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/qczd36g.html,15/05/2022,86934,23938
3 yamaha MT 09,502.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/xsqtct27.html,29/05/2025,109308,39897
4 yamaha MT 09,67425.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/t65gn82.html,22/05/2021,72358,19949
5 yamaha MT 09,10.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/oliavvus.html,27/03/2024,109308,31918
6 yamaha MT 09,10.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/savsj04.html,27/03/2024,129089,31918
7 yamaha MT 09,6420.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/62kdjmc.html,27/02/2022,96814,23938
8 yamaha MT 09,5000.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/dflc17sz.html,15/05/2023,93701,27929
9 yamaha MT 09,5500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09-2024/hc3uv9c.html,29/05/2024,114524,39897
10 yamaha MT 09,9300.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/pgf3642.html,15/05/2023,114524,27929
11 yamaha MT 09,4900.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/62swjgc.html,23/01/2023,110349,27929
12 yamaha MT 09,500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/blequip.html,29/05/2024,93691,39897
13 yamaha MT 09,15550.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/qczd36g.html,15/05/2022,86934,23938
14 yamaha MT 09,10.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/oliavvus.html,27/03/2024,109308,31918
15 yamaha MT 09,5500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09-2024/hc3uv9c.html,29/05/2024,114524,39897
16 yamaha MT 09,4900.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/62swjgc.html,23/01/2023,110349,27929
17 yamaha MT 09,6420.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/62kdjmc.html,27/02/2022,96814,23938
18 yamaha MT 09,6300.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/urvfudy.html,29/05/2023,97866,27929
19 yamaha MT 09,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09-890-y-ant/ursalky.html,10/04/2024,114524,31918
20 yamaha MT 09,16331.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/8ur6rz4.html,24/06/2021,109266,23938
21 yamaha MT 09,12623.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/zck1kf4.html,24/06/2021,114472,23938
22 yamaha MT 09,15000.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/hc3uv7c.html,29/05/2021,96304,23938
23 yamaha MT 09,5000.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/dflc17sz.html,15/05/2023,93701,27929
24 yamaha MT 09,14300.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/sa80te4.html,29/05/2022,83186,23938
25 yamaha MT 09 SP,20000.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09-sp/yamaha-mt-09-sp/k5jnemg.html,29/05/2022,104008,29192
26 yamaha MT 09,5500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/19jdiah.html,29/05/2022,91515,23938
27 yamaha MT 09,24871.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/wa4x161.html,27/03/2022,93691,23938
28 yamaha MT 09,21970.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/mt-09/nejox7k.html,15/04/2023,97814,27929
29 yamaha MT 09,500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/blequip.html,29/05/2024,93691,39897
30 yamaha MT 09,2208.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/y3dehrs.html,29/05/2024,114513,39897
31 yamaha MT 09,500.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/fke891j.html,29/05/2024,93691,39897
32 yamaha MT 09,10.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/g5502d5.html,29/05/2024,104102,39897
33 yamaha MT 09,7980.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/y3doffyz.html,29/05/2022,87358,23938
34 yamaha MT 09,14300.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09/yamaha-mt-09/urvfuty.html,29/05/2022,83186,23938
35 yamaha MT 09 SP,31000.0,ESPAGNE,https://www.leparking-moto.ma/moto-occasion-detail/yamaha-mt-09-sp/yamaha-mt-09-sp-2023/1x08an0.html,29/05/2023,93701,34058

```

FIGURE 5.3 – Exemple de database pour la recherche "yamaha mt 09"

5.5 Représentation graphique des marges brutes

Cette section présente la méthodologie de visualisation des marges brutes calculées pour chaque annonce de moto. Le script génère un diagramme en barres verticales où chaque élément représente une annonce spécifique, avec une codification couleur permettant d'identifier instantanément la rentabilité potentielle.

5.5.1 Structure du graphique

- **Axe horizontal** : Numéro d'identification de l'annonce
- **Axe vertical** : Montant en dirhams marocains (DH)
- **Barres verticales** :
 - Base : Coût total d'acquisition (prix d'achat + frais de dédouanement)
 - Sommet : Prix moyen estimé au Maroc
 - Hauteur : Marge brute potentielle
- **Codification couleur** :
 - **Vert** : Marge positive (bénéfice potentiel)

— Rouge : Marge négative (perte potentielle)

5.5.2 Algorithme de génération

Le script Python procède en 5 étapes clés :

1. Chargement du dataset final contenant tous les calculs
2. Vérification et calcul des marges brutes
3. Filtrage des annonces incomplètes
4. Création des éléments graphiques pour chaque annonce

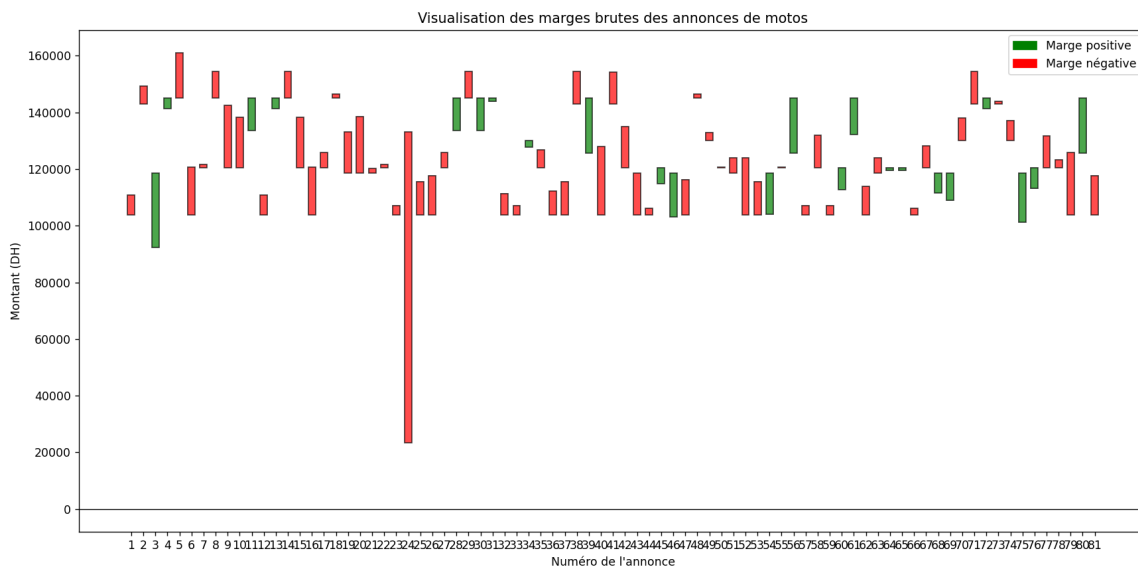


FIGURE 5.4 – Visualisation des marges brutes par annonce de moto

5.5.3 Interprétation des résultats

Cette visualisation permet d'identifier rapidement :

- Les annonces les plus rentables (barres vertes les plus hautes)
- Les annonces à éviter (barres rouges)
- La distribution globale des opportunités
- Les variations de rentabilité entre différents modèles

Le graphique constitue un outil d'aide à la décision pour la sélection des motos présentant le meilleur potentiel économique lors de l'importation.

5.6 Code Python de visualisation

```
# Chargement des données finales
df = pd.read_csv("resultats_avec_prix_maroc.csv")

# Calcul des marges si nécessaire
if 'Marge' not in df.columns or df['Marge'].isnull().all():
    # Calcul du coût total: prix d'achat + frais de dédouanement
```



```
df['Prix d\'achat avec frais'] = df['Prix (DH)'] + df['Valeur taxable']
# Calcul de la marge brute
df['Marge'] = df['Prix Moyen Maroc'] - df['Prix d\'achat avec frais']

# Filtrage des annonces valides
df_valid = df.dropna(subset=['Marge', 'Prix Moyen Maroc', 'Prix d\'achat avec
↪ frais'])
df_valid = df_valid.reset_index(drop=True)

# Préparation des données pour le graphique
indices = np.arange(len(df_valid)) # Numérotation des annonces
colors = ['green' if m >= 0 else 'red' for m in df_valid['Marge']] # Codage couleur

# Création de la figure
fig, ax = plt.subplots(figsize=(14, 7))

# Dessin des barres pour chaque annonce
for i, row in df_valid.iterrows():
    cout_total = row["Prix d'achat avec frais"]
    prix_maroc = row["Prix Moyen Maroc"]
    marge = row["Marge"]

    # Détermination de la base et hauteur de la barre

    .
    .
    .
    .

ax.legend(handles=[positive, negative], loc='upper right')

# Optimisation de l'espace
plt.tight_layout()

# Ajout d'une grille pour meilleure lisibilité
ax.grid(axis='y', linestyle='--', alpha=0.7)

# Sauvegarde et affichage
plt.savefig("marge_candlestick_annonces.png", dpi=300, bbox_inches='tight')
plt.show()
```

Chapitre 6

Description des étapes du scrapping

6.1 Saisie des paramètres de recherche

Cette étape initiale permet à l'utilisateur de définir les critères de recherche pour le scrapping. Le script demande successivement la marque de la moto (ex : yamaha), le modèle (ex : mt 09), le nombre de pages à scraper, et le pays cible (espagne). Une fois ces paramètres saisis, le système génère automatiquement l'URL de navigation vers le site leparking-moto.ma avec les filtres appropriés, préparant ainsi la session de scrapping avec les critères spécifiés par l'utilisateur.

```
DevTools listening on ws://127.0.0.1:57351/devtools/browser/a91323c2-4905-4671-b5e3-1dcad201d37c
Entrez la marque de la moto (ex: yamaha) : yamaha
Entrez le modèle de la moto (ex: mt 07) : mt 09
Combien de pages veux-tu scraper ? : 1
Entrer le pays voulue :espagne
Navigation vers : https://www.leparking-moto.ma/moto-occasion/espagne/yamaha-mt-09.html
```

FIGURE 6.1 – Interface de saisie des paramètres de recherche

6.2 Affichage des résultats de recherche

Cette étape présente la page de résultats après exécution du script de scrapping sur le site leparking-moto.ma. La page affiche une interface de recherche filtrée avec les critères "espagne yamaha mt 09", montrant deux annonces de motos Yamaha MT-09 d'occasion en provenance d'Espagne. Chaque résultat contient les informations essentielles (prix, kilométrage, année, cylindrée) avec une photo et un bouton de détail.

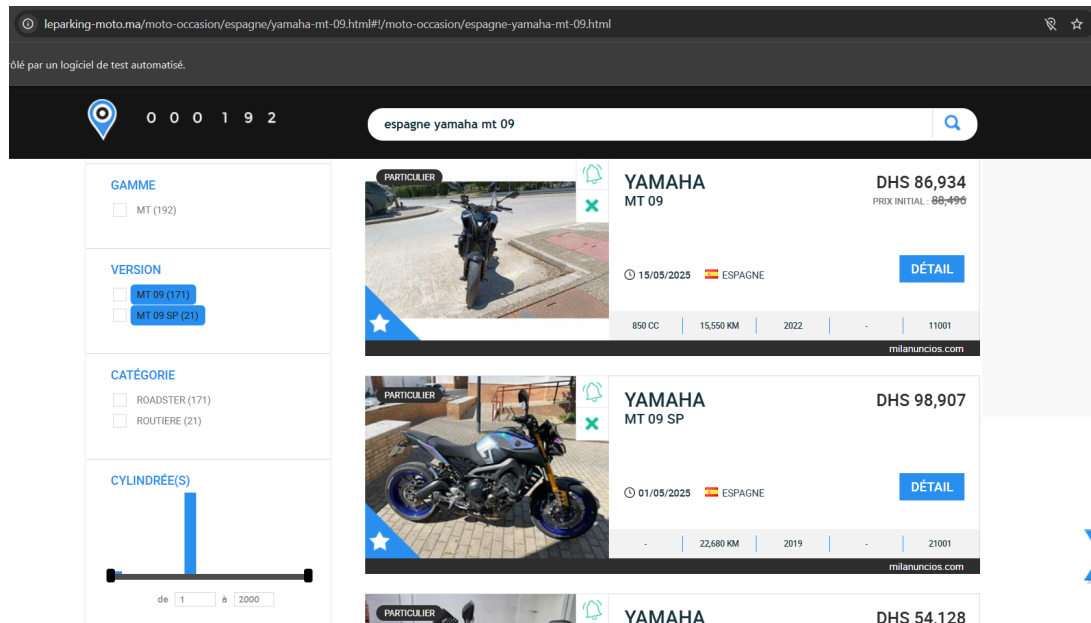


FIGURE 6.2 – Page de résultats de recherche sur leparking-moto.ma

6.3 Navigation vers le calculateur de dédouanement

Pour chaque annonce de moto identifiée, le script ouvre automatiquement le site officiel des douanes marocaines (<https://www2.douane.gov.ma/mcv/>) et navigue vers la section "CALCUL DES DROITS ET TAXES". Cette étape permet d'accéder à l'outil de calcul des frais de dédouanement qui sera utilisé pour estimer les coûts d'importation de chaque véhicule

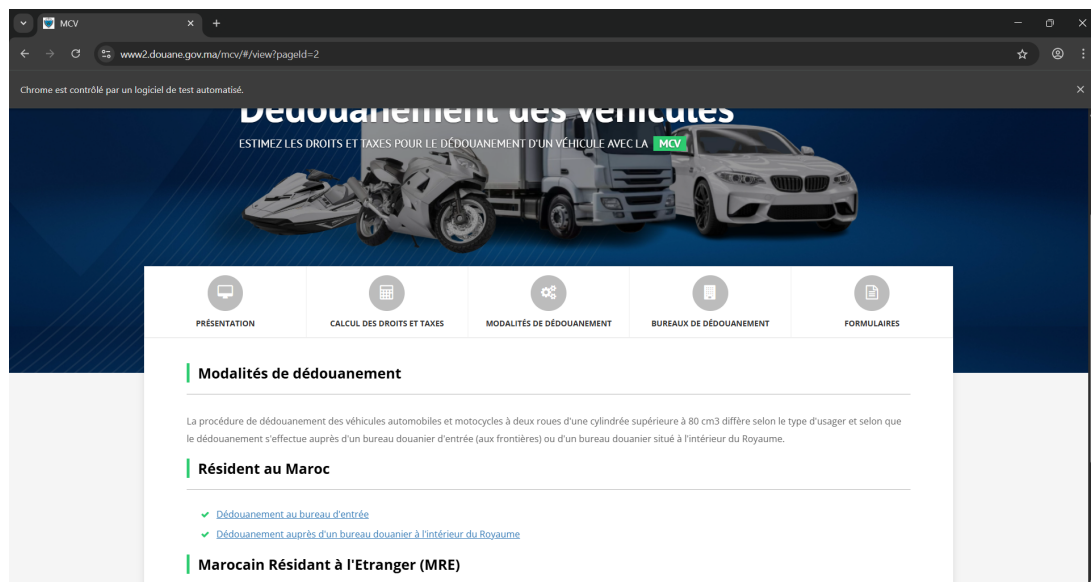


FIGURE 6.3 – Interface du calculateur de dédouanement des douanes marocaines

6.4 Sélection du type de véhicule

Une fois dans la section "Calcul des droits et taxes", le script sélectionne automatiquement la catégorie "MOTOCYCLE" parmi les différents types de véhicules disponibles (Automobile, Motocycle, Jet Ski, Poids Lourd). Cette sélection permet d'accéder aux formulaires de calcul spécifiques aux motocycles et d'appliquer les tarifs douaniers appropriés pour ce type de véhicule.

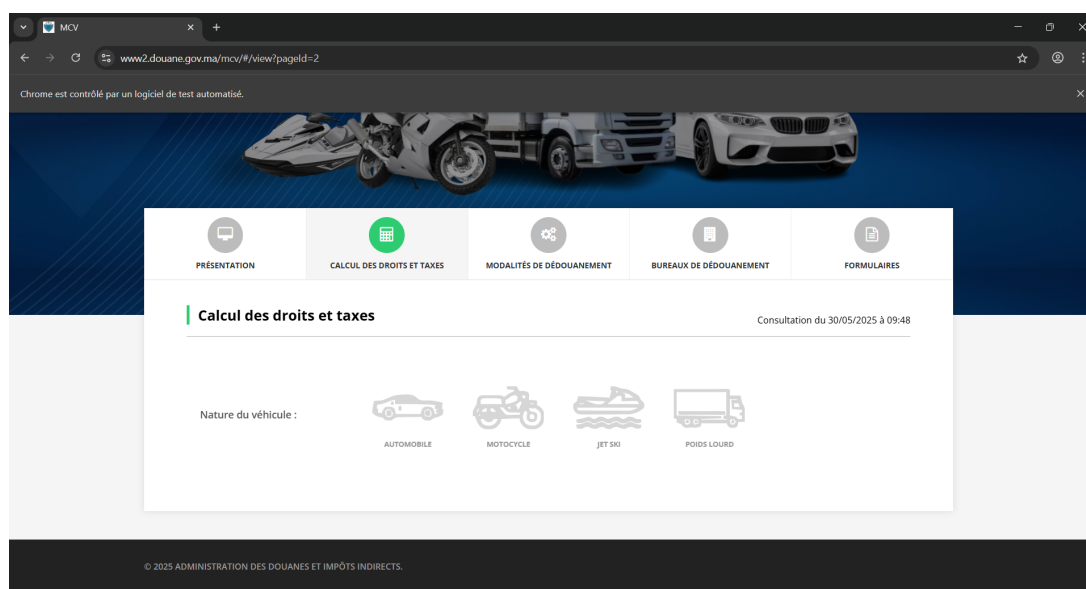


FIGURE 6.4 – Sélection du type de véhicule - Motocycle

6.5 Sélection de la marque du véhicule

Après avoir choisi la catégorie "Motocycle", le système affiche un formulaire de calcul avec plusieurs champs à remplir. Le script procède alors à la sélection de la marque de la moto dans le menu déroulant qui propose une liste exhaustive des marques disponibles (AGM, AJP, AJS, APOLLO, APRILIA, AUSTIN, AVINTO, BEELINE, etc.). Cette étape permet d'identifier précisément le constructeur du véhicule pour appliquer les tarifications douanières correspondantes.

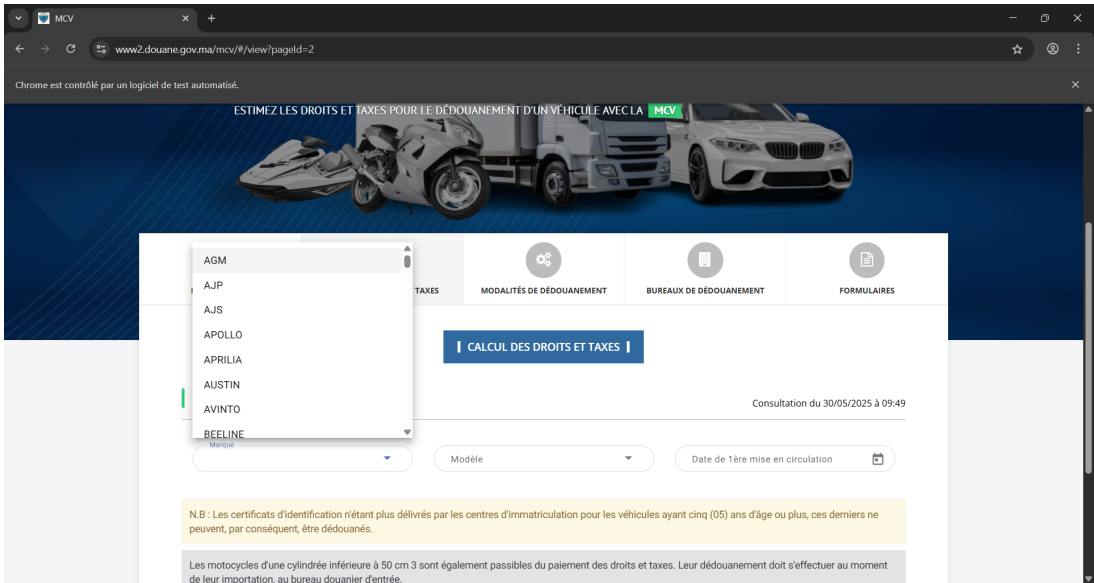


FIGURE 6.5 – Sélection de la marque dans le menu déroulant

6.6 Sélection du modèle du véhicule

Une fois la marque sélectionnée, le script procède à la sélection du modèle spécifique de la moto dans le second menu déroulant "Modèle". Cette liste se met à jour dynamiquement en fonction de la marque choisie précédemment et affiche tous les modèles disponibles pour cette marque dans la base de données des douanes. Cette sélection permet d'affiner davantage les calculs de droits et taxes en fonction des caractéristiques techniques précises du véhicule.

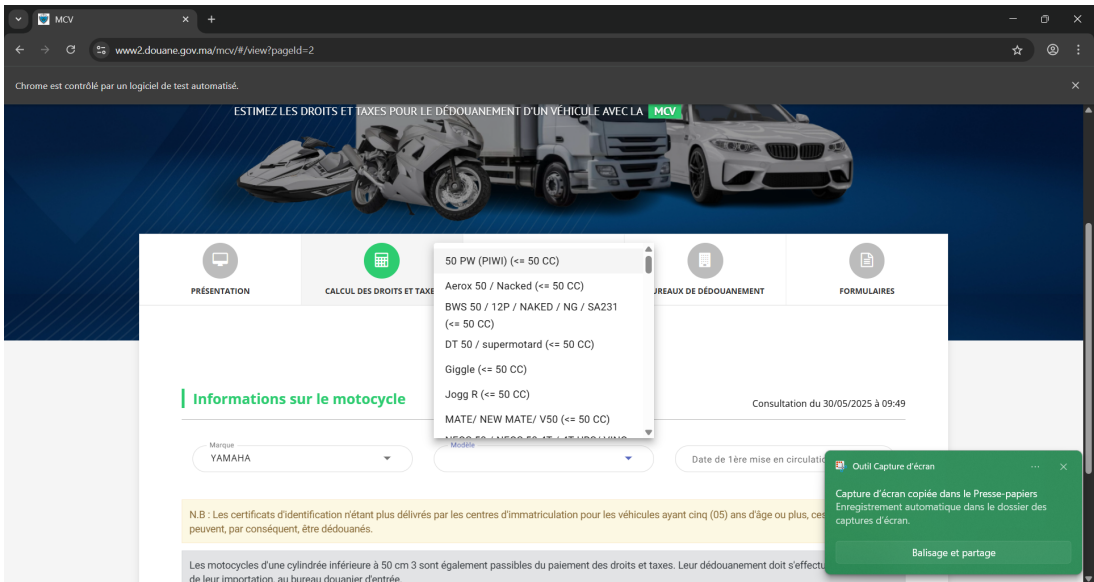


FIGURE 6.6 – Sélection du modèle dans le menu déroulant

6.7 Saisie de la date de première mise en circulation

Après avoir sélectionné la marque et le modèle, le script active le champ "Date de 1ère mise en circulation" qui ouvre un calendrier interactif. Le système sélectionne automatiquement l'année de première mise en circulation de la moto récupérée depuis l'annonce, puis le mois et enfin le jour. Cette information est cruciale pour le calcul des droits de douane car l'âge du véhicule influence directement les tarifs appliqués et détermine les conditions de dédouanement.

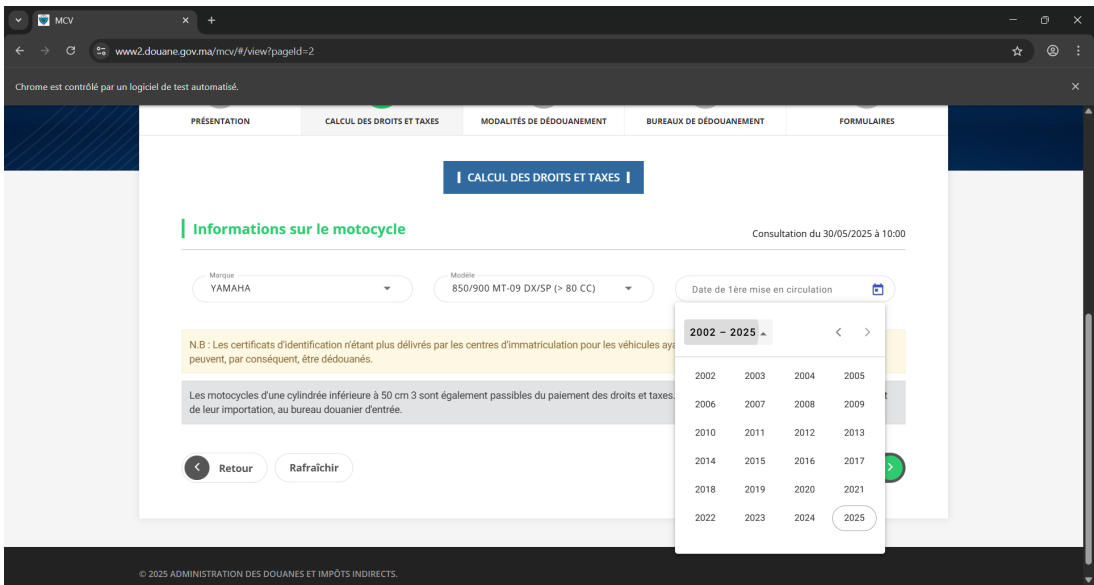


FIGURE 6.7 – Ouverture du calendrier pour la sélection de la date

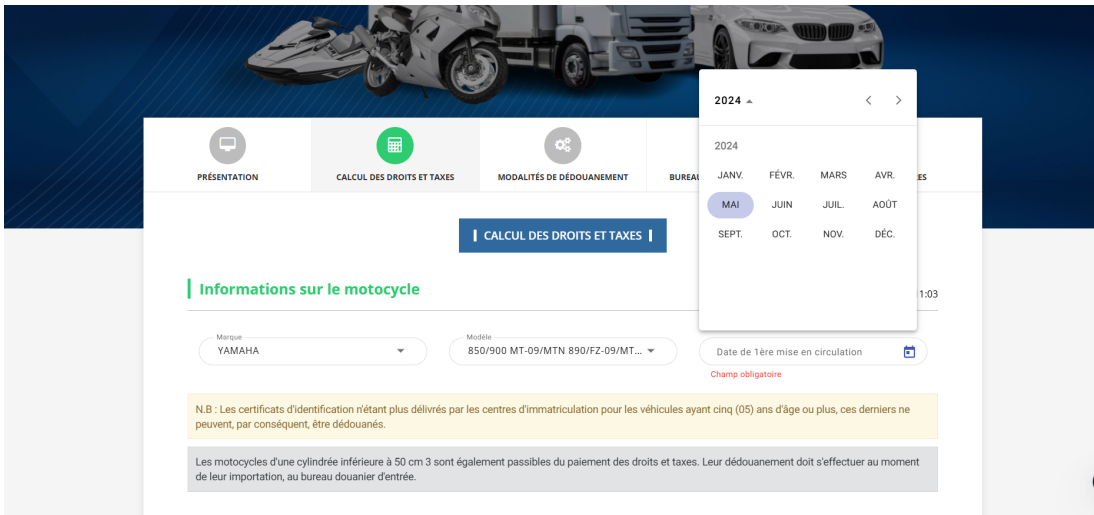


FIGURE 6.8 – Sélection de l'année dans le calendrier

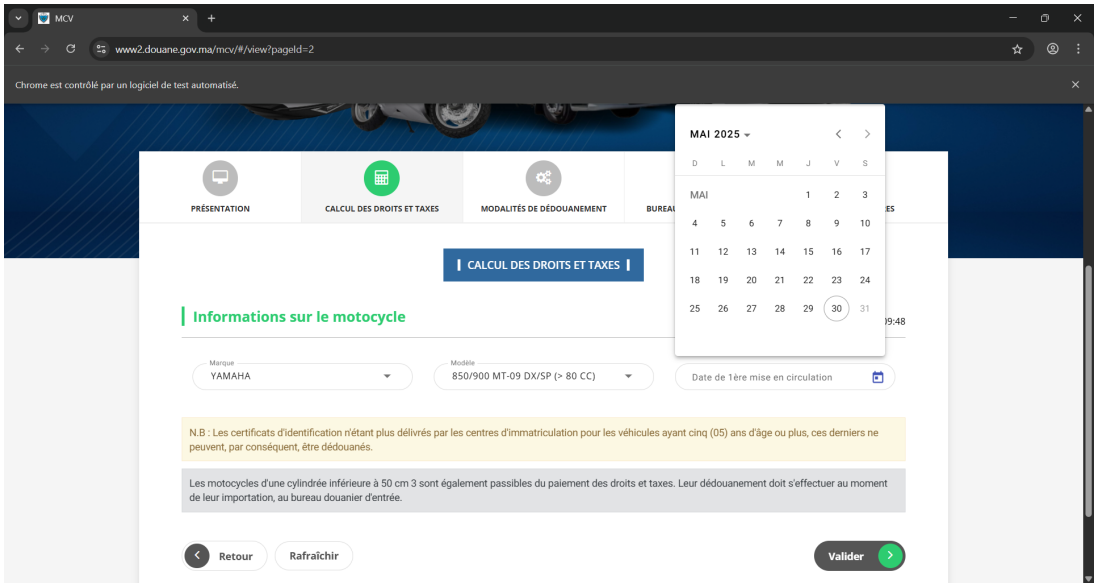


FIGURE 6.9 – Sélection du mois et du jour dans le calendrier

6.8 Extraction des frais totaux de dédouanement

Après avoir soumis tous les paramètres du véhicule (marque, modèle, date de mise en circulation), le script récupère automatiquement le montant total des frais de dédouanement affiché sur la page de résultats. Le système localise la section "Montant total des droits et des taxes" qui présente le montant global des droits et taxes exigibles pour l'importation du véhicule au Maroc. Cette valeur inclut l'ensemble des composantes douanières (droit d'importation, TVA, taxe parafiscale, etc.) calculées par la plateforme des douanes

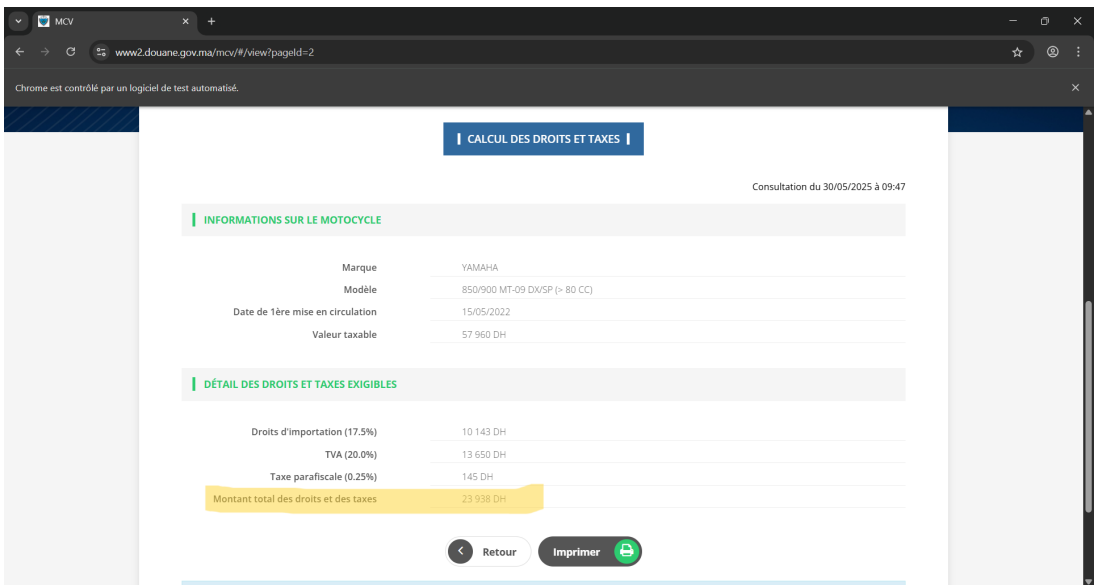


FIGURE 6.10 – Extraction du montant total des frais de dédouanement

6.9 Scrapping comparatif des annonces locales (Avito Maroc)

Une fois le scrapping des annonces de motos d'occasion en provenance d'Europe effectué (ex. Espagne, France, etc.), le script poursuit automatiquement avec une seconde phase de scrapping ciblée : il interroge la plateforme Avito.ma afin d'identifier le prix moyen des motos équivalentes déjà disponibles sur le marché marocain.

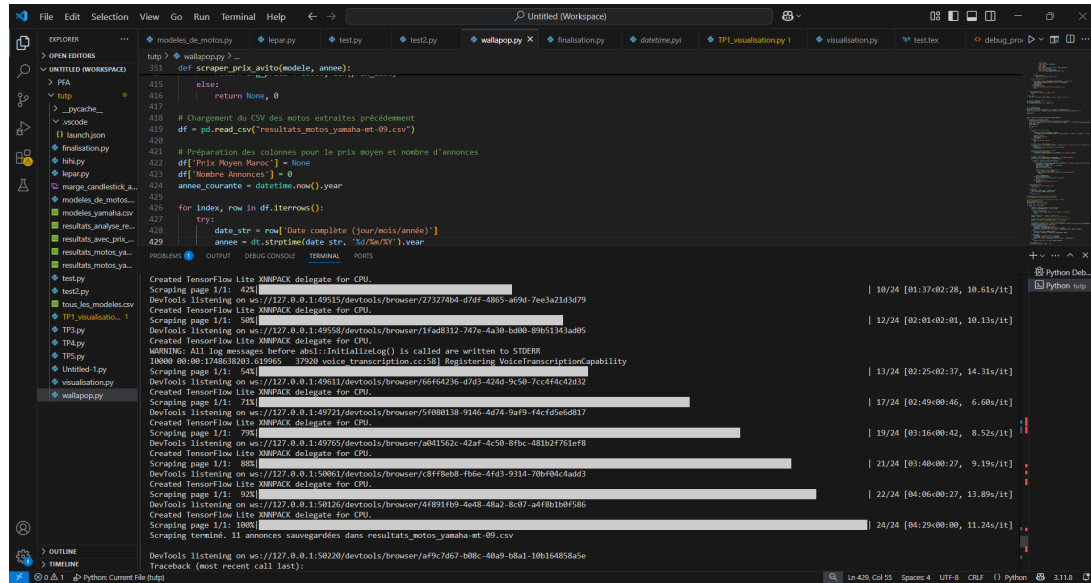


FIGURE 6.11 – Finalisation du scrapping des annonces en Europe

6.9.1 Critères de correspondance

Le scrapping Avito est déclenché en utilisant les critères suivants, issus des annonces européennes :

- Marque de la moto (ex : Yamaha)
- Modèle (ex : MT-09)
- Année de première mise en circulation (ex : 2019)

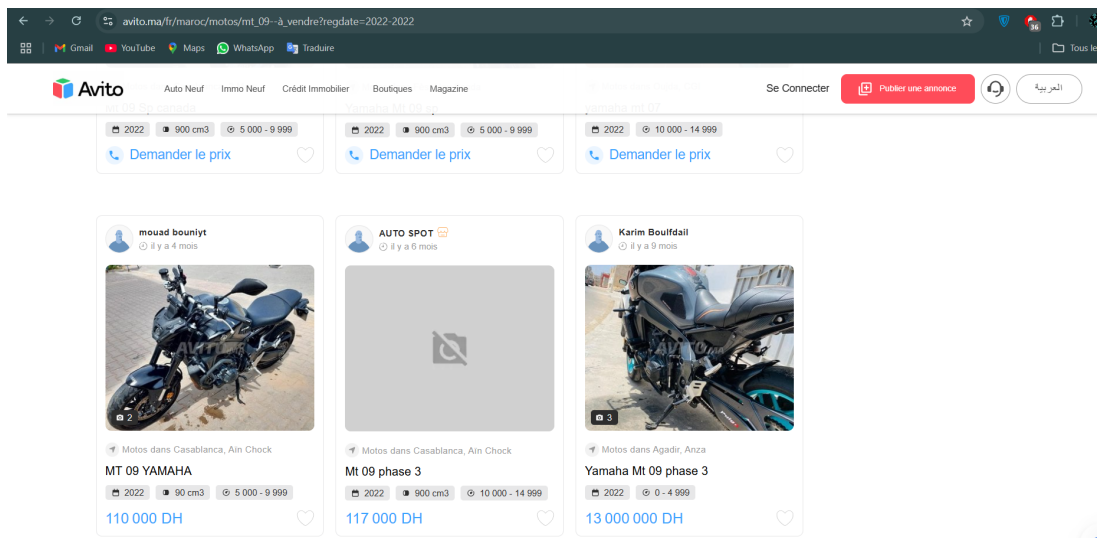


FIGURE 6.12 – Scrapping des annonces de la même moto au Maroc

6.10 Visualisation finale des marges brutes

À l'issue du scrapping et après avoir sauvegardé dans un fichier CSV toutes les données nécessaires (prix d'achat, frais de dédouanement, estimation du prix de vente au Maroc), le script réalise une analyse comparative des marges brutes.

6.10.1 Diagramme de type "bougie" des annonces

- Chaque annonce est représentée par une bougie verticale illustrant la rentabilité :
- En vert : les annonces dont la marge brute est positive, donc rentables.
- En rouge : les annonces non rentables, avec une marge négative.

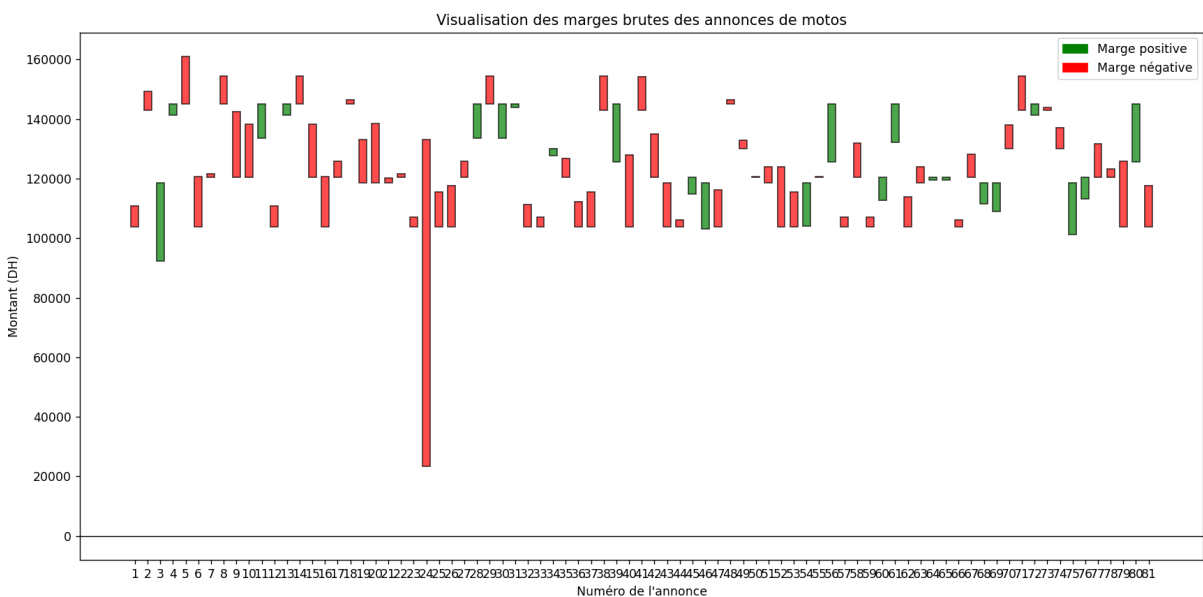


FIGURE 6.13 – Visualisation des marges brutes des annonces de motos

6.10.2 Lecture du graphique

Ce graphique permet à l'utilisateur :

- D'identifier rapidement les meilleures opportunités d'importation.
- D'éliminer les annonces non rentables avant prise de décision.
- De visualiser l'écart entre coût total et estimation de revente.

Chaque barre verticale est centrée sur le coût total (incluant achat + port + douane) et sa hauteur indique l'amplitude de la marge. Cette visualisation permet une prise de décision éclairée et constitue une aide à l'importation stratégique.

Chapitre 7

Conclusion et perspectives

Conclusion



Ce projet a permis de développer un système complet d'aide à la décision pour l'importation de motos d'occasion, combinant des techniques avancées de **web scraping**, d'analyse comparative, et de visualisation graphique. En automatisant la collecte des annonces depuis plusieurs sources (sites européens et marocains), en intégrant le simulateur des droits de douane du Maroc, et en mettant en œuvre une méthodologie d'estimation des marges brutes, le système fournit un outil efficace pour identifier les opportunités d'achat rentables.

Les résultats obtenus sont convaincants : à travers une représentation visuelle claire des marges positives et négatives, l'utilisateur peut prendre des décisions rapides et basées sur des données réelles, réduisant considérablement les risques liés à l'importation. Le projet se positionne ainsi comme une solution à forte valeur ajoutée pour les particuliers et professionnels du secteur automobile ou de la logistique.

7.1 Perspectives

Afin d'augmenter la robustesse, l'intelligence et l'impact économique du système développé, plusieurs pistes d'amélioration ambitieuses peuvent être envisagées :

7.1.1 1. Intégration de l'intelligence artificielle pour la prédiction des marges futures

Plutôt que de se limiter à une estimation statique, il serait pertinent de développer un module de **machine learning** (ex : régression, forêt aléatoire) permettant de prédire les marges futures en fonction de l'évolution du marché, des tendances saisonnières, ou des fluctuations monétaires. Ce modèle pourrait apprendre à partir d'un historique enrichi d'annonces pour détecter les meilleurs moments pour importer.

7.1.2 Déploiement d'une plateforme web interactive

Transformer ce projet en une véritable **application web full-stack** accessible au public serait une étape majeure. L'utilisateur pourrait y saisir ses préférences, visualiser les résultats en temps réel (Streamlit, Dash, ou React + Flask), consulter des recommandations personnalisées et recevoir des alertes lorsque des motos deviennent rentables. Un espace administrateur permettrait aussi de gérer les paramètres et surveiller les performances du scrapping.

7.1.3 Enrichissement de la base de données avec d'autres marchés

Étendre le scrapping sur d'autres sites européens (Wallapop, Leboncoin...) permettrait d'élargir le champ de comparaison. Parallèlement, inclure d'autres plateformes marocaines (MarocAnnonces, Moteur.ma...) renforcerait la précision de l'estimation du prix local. À terme, une **base de données centralisée multi-pays** enrichie quotidiennement constituerait un atout majeur.

7.1.4 Système d’alerte intelligente et scoring

Un module de **scoring automatique** pourrait classer les annonces selon plusieurs critères : état du véhicule, âge, rentabilité, confiance dans la source, etc. Couplé à un système d’alerte (email/SMS/app) en cas de bonne affaire détectée, cela transformerait l’outil en véritable assistant personnel d’investissement.

7.1.5 Automatisation des démarches administratives

À plus long terme, le système pourrait pré-remplir les formulaires de douane, préparer les documents nécessaires à l’importation et même simuler les délais de transport. En connectant cette solution avec les services logistiques (transitaire, assurance, dédouanement), elle deviendrait une **chaîne d’importation semi-automatisée**.

7.1.6 Couplage avec une blockchain de traçabilité des véhicules

Enfin, pour garantir la fiabilité des informations (kilométrage réel, entretiens, accidents), le système pourrait être interfacé avec une **blockchain de traçabilité des véhicules** importés. Chaque importation validée serait enregistrée dans un registre infalsifiable, assurant la transparence du processus jusqu’au client final.

Conclusion Générale

Ce projet s’inscrit dans un contexte économique où l’importation de véhicules d’occasion, en particulier de motos, représente une opportunité commerciale significative. En développant une solution complète d’analyse automatisée des opportunités d’importation, nous avons répondu à un besoin réel : disposer d’un outil capable de traiter un grand volume d’annonces, de comparer les coûts d’importation avec la valeur réelle du marché local, et d’identifier rapidement les options les plus rentables.

La première phase a consisté à automatiser le **scraping des annonces de motos** depuis des plateformes européennes telles que *LeParking-Moto* [5] et *Motoconcess.fr*, en tenant compte des paramètres saisis par l’utilisateur (marque, modèle, pays, année). Ensuite, un second module de scraping ciblait le site *Avito.ma* [2] afin d’estimer le prix moyen local d’une moto équivalente sur le marché marocain.

En parallèle, le système interroge automatiquement la plateforme officielle de la **douane marocaine** [1] pour obtenir une estimation des droits et taxes à l’importation. Ce processus génère ainsi, pour chaque annonce, une fiche complète contenant les données du véhicule, les coûts estimés d’importation, et le potentiel de revente local.

Une dernière phase permet la visualisation graphique des marges brutes sous forme de diagrammes en bougie, ce qui facilite grandement l’évaluation de la rentabilité. Grâce à une codification couleur intuitive (vert pour les marges positives, rouge pour les négatives), l’utilisateur peut rapidement repérer les opportunités les plus intéressantes.

Sur le plan technique, ce projet a mobilisé plusieurs outils puissants : **Selenium** [9] pour l’automatisation du scraping, **Pandas** [6] pour le traitement structuré des données, et **Matplotlib** [3] pour la génération des graphiques analytiques. Le développement s’est effectué dans l’environnement **Visual Studio Code** [7], et le rapport a été entièrement rédigé en **L^AT_EX** [4] sur la plateforme collaborative *Overleaf* [8].

Au-delà de sa mise en œuvre actuelle, le projet ouvre de nombreuses **perspectives** :

- l’enrichissement des sources de données internationales,
- l’intégration d’un moteur de prédiction intelligent basé sur des modèles de machine learning,
- le déploiement d’une plateforme web interactive,
- l’automatisation des démarches douanières,
- et à plus long terme, l’usage de la blockchain pour certifier la traçabilité des véhicules importés.

En conclusion, ce projet constitue un socle solide pour de futures innovations dans le domaine de l’importation intelligente. Il démontre concrètement comment la technologie peut transformer un processus complexe et incertain en un système rigoureux, efficace et exploitable à grande échelle.

Bibliographie

- [1] Administration des Douanes Marocaines. Calculateur en ligne des droits et taxes, 2025. Consulté en mai 2025.
- [2] Avito.ma. Plateforme d’annonces de véhicules d’occasion au maroc, 2025. Consulté en mai 2025.
- [3] John D. Hunter. Matplotlib : A 2d graphics environment. *IEEE Computing in Science & Engineering*, 9(3), 2007.
- [4] Leslie Lamport. *LT_EX : A Document Preparation System*. Addison Wesley, Massachusetts, 2 edition, 1994.
- [5] LeParking-Moto.ma. Annonces de motos d’occasion à l’international, 2025. Consulté en mai 2025.
- [6] Wes McKinney. *Python for Data Analysis*. O’Reilly Media, 2 edition, 2018.
- [7] Microsoft. Visual studio code - code editing. redefined, 2025. Consulté en mai 2025.
- [8] Overleaf. Overleaf, online latex editor, 2025. Consulté en mai 2025.
- [9] SeleniumHQ. *Selenium WebDriver Documentation*, 2025. Consulté en mai 2025.

Annexes

Annexe Code 1

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException, NoSuchElementException
.
.
.

import traceback
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from datetime import date, timedelta

# ==== CONFIGURATION SELENIUM ====
options = Options()
options.add_argument("--window-size=1280x800")
options.add_argument("--ignore-certificate-errors")
.
.
.

options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)
↳ AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36")
options.add_experimental_option("prefs", {
    "profile.default_content_setting_values.geolocation": 2
})

df_modeles = pd.read_csv("tous_les_modeles.csv")

def normalize(text):
    return re.sub(r"[^\w]", "", text.lower())
    # ==== CHROME DRIVER ====
service = Service(r"C:\chromedriver-win64\chromedriver.exe")
driver = webdriver.Chrome(service=service, options=options)

# ==== DEMANDER LA MOTO ET PAGES ====
marque = input("Entrez la marque de la moto (ex: yamaha) : ").strip().lower()
modele = input("Entrez le modèle de la moto (ex: mt 07) : ").strip().lower()
moto = marque + " " + modele
moto_url = moto.replace(" ", "-")
nb_pages = int(input("Combien de pages veux-tu scraper ? : "))

pays=input("Entrer le pays voulue :").strip().lower()

url = f"https://www.leparking-moto.ma/moto-occasion/{pays}/{moto_url}.html"
print(f"Navigation vers : {url}")

driver.get(url)
```



```
time.sleep(3)

data = []
page_count = 0

# ==== DATE LIMITE (aujourd'hui - 5 ans) ====
date_limite = datetime.date.today() - datetime.timedelta(days=5*365)
```

Annexe Code 2

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
import csv
import time

chemin_chromedriver = r"C:\chromedriver-win64\chromedriver.exe"
url = "https://www2.douane.gov.ma/mcv/#/view?pageId=2"

options = Options()
options.add_argument("--start-maximized")

driver = webdriver.Chrome(service=Service(chemin_chromedriver), options=options)
wait = WebDriverWait(driver, 15)

driver.get(url)

# Ouvrir le menu des marques
wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
↪ "mat-select[formcontrolname='marque']"))).click()
time.sleep(1)

# Récupérer toutes les marques disponibles
wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, "mat-option span"))
marque_elements = driver.find_elements(By.CSS_SELECTOR, "mat-option span")
marques = [m.text.strip() for m in marque_elements if m.text.strip()]

print(f" {len(marques)} marques détectées.")

with open("tous_les_modeles.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Marque", "Modèle"])

    for marque in marques:
        try:
            # Fermer backdrop s'il est présent
            try:
                backdrop = driver.find_element(By.CSS_SELECTOR,
↪ ".cdk-overlay-backdrop")
                driver.execute_script("arguments[0].click();", backdrop)
                time.sleep(0.5)
```

```
except:
    pass

# Réouvrir le menu des marques
wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
    ↳ "mat-select[formcontrolname='marque']"))).click()
time.sleep(1)

    .
    .
    .

for modele in modeles:
    writer.writerow([marque, modele])

except Exception as e:
    print(f" Problème avec {marque} : {e}")
    continue

print(" Export terminé → fichier : tous_les_modeles.csv")
driver.quit()
```

Annexe Code 3

```
import pandas as pd

df_modeles = pd.read_csv("tous_les_modeles.csv")
```

Annexe Code 4

```
def trouver_modele_proche(modele_scrape, marque_scrape, df_modeles):
    modele_norm = normalize(modele_scrape)
    subset = df_modeles[df_modeles["Marque"].str.lower() == marque_scrape.lower()]
    modeles_dispo = subset["Modèle"].dropna().unique()

    # Correspondance manuelle partielle
    for modele in modeles_dispo:
        if normalize(modele_norm) in normalize(modele):
            return modele
        if normalize(modele) in normalize(modele_norm):
            return modele

    return None
```

Annexe Code 5

```
def get_valeur_taxable_total(marque, modele, date_str):

    service = Service(r"C:\chromedriver-win64\chromedriver.exe")
```

```
options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(service=service, options=options)
wait = WebDriverWait(driver, 15)

try:
    jour, mois, annee = date_str.split("/")
    mois_map = {
        "01": "JANV.", "02": "FÉVR.", "03": "MARS", "04": "AVR.", "05": "MAI",
        "06": "JUIN", "07": "JUIL.", "08": "AOÛT", "09": "SEPT.",
        "10": "OCT.", "11": "NOV.", "12": "DÉC."
    }
    mois_nom = mois_map[mois]

    driver.get("https://www2.douane.gov.ma/mcv/#/view?pageId=2")
    time.sleep(2)

    # Clic sur "Calcul des droits"
    onglets = wait.until(EC.presence_of_all_elements_located(
        (By.CSS_SELECTOR,
         ↪ "div.b-detail__main-info-characteristics-one.calculator")))
    for onglet in onglets:
        if "CALCUL DES DROITS" in onglet.text.upper():
            driver.execute_script("arguments[0].click();", onglet)
            break
    time.sleep(1.5)

    # Clic sur "Motocycle"
    moto_svg = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR,
        ↪ "label[for='type1']")))
    driver.execute_script("arguments[0].click();", moto_svg)
    time.sleep(1.5)

    # Sélection de la marque
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
        ↪ "mat-select[formcontrolname='marque']"))).click()
    time.sleep(1)
    marques = wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR,
        ↪ "mat-option span"))))
    for m in marques:
        if m.text.strip().upper() == marque.upper():
            m.click()
            break
    time.sleep(1)

    # Sélection du modèle
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
        ↪ "mat-select[formcontrolname='modele']"))).click()
    time.sleep(1)
    modeles = wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR,
        ↪ "mat-option span"))))
    for mod in modeles:
        if modele.lower() in mod.text.strip().lower():
            mod.click()
            break
    time.sleep(1)
```

```
# Calendrier : ouverture et sélection de la date
wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
↪ "button[aria-label='Open calendar']")))).click()
time.sleep(1)

wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
↪ "button.mat-calendar-period-button")))).click()
time.sleep(1)

# Sélection de l'année avec navigation si nécessaire
while True:
    .
    .
    .
    .

# Extraction de la valeur taxable
elements = driver.find_elements(By.CSS_SELECTOR,
↪ "div.b-compare__block-inside-value")
montant = None
for el in reversed(elements):
    texte = el.text.strip()
    if "DH" in texte:
        montant_str = re.sub(r"[^\d]", "", texte)
        if montant_str:
            montant = int(montant_str)
            break

return montant

except Exception as e:
    print(" Erreur :", repr(e))
    traceback.print_exc()
    return None

finally:
    time.sleep(3)
    driver.quit()
```

Annexe Code 6

```
data = []
page_count = 0

# Date limite (aujourd'hui - 5 ans)
date_limite = datetime.date.today() - datetime.timedelta(days=5*365)

while page_count < nb_pages:
    try:
        annonces = WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CSS_SELECTOR, "li.li-result"))
        )
```

```
for annonce in tqdm(annonces, desc =f"Scraping page {page_count +
↪ 1}/{nb_pages}"):
    try:
        # Extraction du prix
        prix_elements = annonce.find_elements(By.CSS_SELECTOR,
        ↪ "div.price-block p.prix")
        prix_brut = None
        for prix_element in prix_elements:
            texte = prix_element.text.strip()
            if texte:
                prix_brut = texte
                break

        if not prix_brut:
            continue

        prix_nettoye = re.sub(r"^[^d]", "", prix_brut)
        if prix_nettoye:
            prix_num = int(prix_nettoye)
        else:
            continue

        # Extraction du modèle et titre
        try:
            modele_ = annonce.find_element(By.CSS_SELECTOR,
            ↪ "span.sub-title.title-block
            ↪ span.nowrap.title-block").text.strip()
            modele__ = trouver_modele_proche(modele_,marque.upper(),
            ↪ df_modeles)
            titre = f"{marque} {modele_}"
        except:
            titre = "Non précisé"

        # Extraction du lien
        .
        .
        .
        .

        # Ajout des données extraites
        data.append({
            "Titre": titre,
            "Prix (DH)": prix_num,
            "Kilométrage (km)": kilometre,
            "Lieu": lieu,
            "Lien": lien,
            "Date complète (jour/mois/année)": date_circulation,
            "Valeur taxable": get_valeur_taxable_total(marque, modele__,
            ↪ date_circulation)
        })

    except Exception as e:
        print(f"Erreur sur une annonce : {e}")

# Passage à la page suivante
try:
    next_button = driver.find_element(By.CSS_SELECTOR, "li.btn-next a")
```

```
        driver.execute_script("arguments[0].click();", next_button)
        time.sleep(3)
        page_count += 1
    except Exception as e:
        print("Pas de bouton suivant ou clic impossible :", e)
        break

except Exception as e:
    print(f"Erreur pendant le scraping : {e}")
    break
```

Annexe Code 7

```
for d in data:
    d["Lien"] = d["Lien"].strip().lower() # Normalisation du lien

df = pd.DataFrame(data)
df.drop_duplicates(subset="Lien", inplace=True)

df.to_csv(f"resultats_motos_{moto_url}.csv", index=False)
print(f" Export terminé : resultats_motos_{moto_url}.csv")
```

Annexe Code 8

```
driver = webdriver.Chrome(service=service, options=options)

def scraper_prix_avito(modele, annee):
    url = f"https://www.avito.ma/fr/maroc/motos/{modele.lower().replace(' ' , '_')}--%C3%A0_vendre?regdate={annee}-{annee}"
    print(f"URL: {url}")
    driver.get(url)

    prix_list = []

    try:
        # Accepter les cookies si popup
        try:
            cookie_btn = WebDriverWait(driver, 5).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Accepter') or contains(., 'J\'accepte')]"))
            )
            cookie_btn.click()
            print("Cookies acceptés")
            time.sleep(1)
        except:
            print("Pas de popup cookies")
            pass

        .
        .
        .
        .
```

```
        prix_list.append(prix)
    except NoSuchElementException:
        print("Élément prix non trouvé dans une annonce")
        continue
    except Exception as e:
        print(f"Erreur extraction prix : {e}")
        continue

except Exception as e:
    print(f"Erreur lors du scraping : {e}")
    return None, 0

if prix_list:
    avg_price = sum(prix_list) / len(prix_list)
    return avg_price, len(prix_list)
else:
    return None, 0

# Chargement du CSV des motos extraites précédemment
df = pd.read_csv("resultats_motos_{marque}-{modele}.csv")

# Préparation des colonnes pour le prix moyen et nombre d'annonces
df['Prix Moyen Maroc'] = None
df['Nombre Annonces'] = 0
annee_courante = dt.now().year

for index, row in df.iterrows():
    try:
        date_str = row['Date complète (jour/mois/année)']
        annee = dt.strptime(date_str, '%d/%m/%Y').year

        if annee > annee_courante:
            print(f" Année {annee} dans le futur - passage à la suivante")
            continue

        titre = row['Titre']

        .
        .
        .
        .

# Sauvegarde finale
df.to_csv("resultats_avec_prix_maroc.csv", index=False)

# Résumé statistique
if 'Prix Moyen Maroc' in df.columns and df['Prix Moyen Maroc'].notna().any():
    print("\nRésumé statistique :")
    print(f"- Prix moyen au Maroc : {df['Prix Moyen Maroc'].mean():.2f} DH")
    print(f"- Marge moyenne : {df['Marge'].mean():.2f} DH")
    print(f"- Rentabilité moyenne : {df['Rentabilité %'].mean():.2f} %")
else:
    print("\n Aucune donnée de prix disponible pour calculer les statistiques")
```

```
print("\n Données sauvegardées dans resultats_avec_prix_maroc.csv")

df = pd.read_csv("resultats_avec_prix_maroc.csv")

# Calculer la marge brute si pas déjà présente
if 'Marge' not in df.columns or df['Marge'].isnull().all():
    df['Prix d\'achat avec frais'] = df['Prix (DH)'] + df['Valeur taxable']
    df['Marge'] = df['Prix Moyen Maroc'] - df['Prix d\'achat avec frais']

# Filtrer les annonces valides avec marge calculée
df_valid = df.dropna(subset=['Marge', 'Prix Moyen Maroc', 'Prix d\'achat avec
↪ frais']).reset_index(drop=True)
```

Annexe Code 9

```
# Chargement des données finales
df = pd.read_csv("resultats_avec_prix_maroc.csv")

# Calcul des marges si nécessaire
if 'Marge' not in df.columns or df['Marge'].isnull().all():
    # Calcul du coût total: prix d'achat + frais de dédouanement
    df['Prix d\'achat avec frais'] = df['Prix (DH)'] + df['Valeur taxable']
    # Calcul de la marge brute
    df['Marge'] = df['Prix Moyen Maroc'] - df['Prix d\'achat avec frais']

# Filtrage des annonces valides
df_valid = df.dropna(subset=['Marge', 'Prix Moyen Maroc', 'Prix d\'achat avec
↪ frais'])
df_valid = df_valid.reset_index(drop=True)

# Préparation des données pour le graphique
indices = np.arange(len(df_valid)) # Numérotation des annonces
colors = ['green' if m >= 0 else 'red' for m in df_valid['Marge']] # Codage couleur

# Création de la figure
fig, ax = plt.subplots(figsize=(14, 7))

# Dessin des barres pour chaque annonce
for i, row in df_valid.iterrows():
    cout_total = row["Prix d'achat avec frais"]
    prix_maroc = row["Prix Moyen Maroc"]
    marge = row["Marge"]

    # Détermination de la base et hauteur de la barre
    .
    .
    .
    .

ax.legend(handles=[positive, negative], loc='upper right')

# Optimisation de l'espace
plt.tight_layout()
```



```
# Ajout d'une grille pour meilleure lisibilité
ax.grid(axis='y', linestyle='--', alpha=0.7)

# Sauvegarde et affichage
plt.savefig("marge_candlestick_annonces.png", dpi=300, bbox_inches='tight')
plt.show()
```